

Università Politecnica delle Marche

Corso di Laurea Magistrale

Ingegneria Informatica e dell'Automazione



Sistemi Operativi Dedicati

Studenti

Accattoli Eleonora - S1106361

Brugiavini Alessio - S1106565

Anno Accademico 2022/2023

Indice

1	Introduzione	5
2	Strumenti e software utilizzati	6
3	Sviluppo Progetto	7
3.1	Collegamento dispositivi	7
3.2	Configurazione Raspberry	8
3.3	Librerie	16
3.4	FreeRTOS	19
3.5	ESP32, MQTT Broker e MQTT Client	22
3.6	Sensori	23
3.6.1	Sensor Task	23
3.6.2	MQTT Task	25
3.6.3	SyncTimeTask	25
3.7	Bot Telegram	26
3.8	Creazione Database e caricamento dati	27
3.8.1	Sviluppo Script	29
3.9	Grafici	36
4	Considerazioni finali	46

Elenco delle figure

1	Struttura progetto	5
2	Schema di cablatura del progetto	8
3	Raspberry Pi Imager	9
4	Abilitazione SSH	10
5	RealVNC Viewer	11
6	Modifica connessione	12
7	Configurazione IP statico	12
8	Visualizzazione IP	13
9	Visualizzazione IP da terminale	13
10	File Service MQTT	15
11	File Service Bot	15
12	Stato MQTT Broker Service	16
13	Stato Telegram Bot Service	16
14	File requirements.txt	19
15	Bot Telegram	27
16	GUI Xampp	28
17	File JSON	30
18	Messaggio di errore	34
19	Messaggio di errore	34
20	Richiesta comando leggi	40
21	Grafico andamento temperatura	41
22	Grafico andamento pressione	41
23	Grafico andamento luminosità	42
24	Richiesta comando leggi	43
25	Grafico andamento temperatura	44
26	Grafico andamento pressione	44
27	Grafico andamento luminosità	45

Listings

1	Creazione Task	19
2	Rilevazione sensori e sincronizzazione data e ora	20
3	Inizializzazione sensori	21
4	NTP Client	21
5	Connessione WiFi	21
6	Dichiarazione costanti e variabili globali	21
7	Connessione MQTT Broker	22
8	Connessione WiFi e MQTT Broker	22
9	Sottoscrizione topic	23
10	Struct SensorData	24
11	Task Lettura dati	24
12	Invio dati alla coda	25
13	Ricezione e pubblicazione dati	25
14	Task sincronizzazione data e ora	26
15	Sincronizzazione NTP	26
16	Dati configurazione database	29
17	Connessione al database	29
18	Lettura file JSON	30
19	Lettura file JSON e connessione database	30
20	MQTT Client	31
21	Funzione ricezione messaggi MQTT	32
22	Controllo valori	32
23	Funzione invio alert	33
24	Funzione send message bot	33
25	Comando start	34
26	Comando leggi con estrazione dati	35
27	Comando leggi con estrazione dati	36
28	Calcolo statistiche	36
29	Definizione istruzioni main	38

1 Introduzione

L'obiettivo del progetto consiste nel creare un sistema in grado di rilevare e comunicare in tempo reale i dati rilevati di temperatura, pressione e luminosità dai sensori nell'ambiente. Il sistema sarà composto da:

- **Raspberry Pi 4B**
- **ESP32**: microcontrollore utilizzato per la progettazione di dispositivi connessi e di sistemi IoT.
- Sensore di temperatura e pressione **BMP280**
- Sensore di luminosità **BH1750**
- Modulo RTC **PCF8523**
- **Broker e Client MQTT**
- **Bot Telegram**

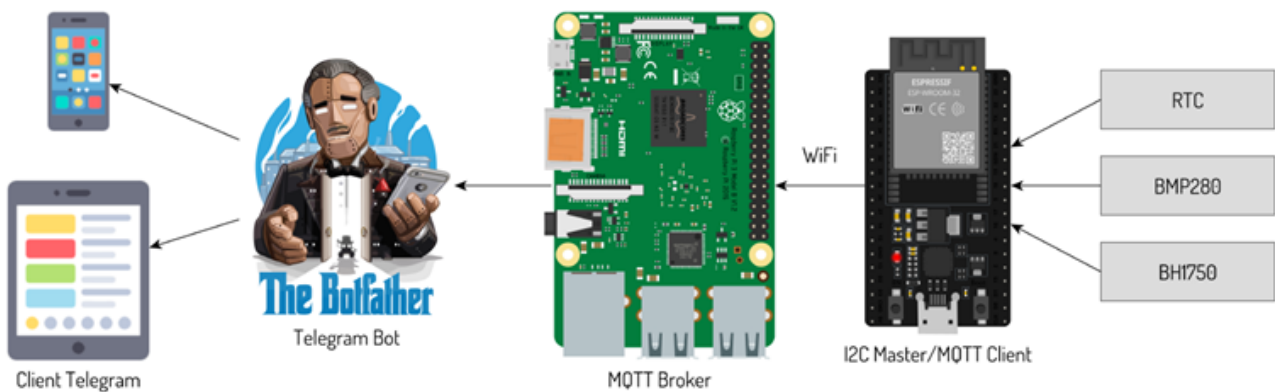


Figura 1: Struttura progetto

I due sensori ed il modulo RTC vengono collegati fisicamente all'ESP32, il quale si occuperà di trasmettere i dati rilevati dai sensori al Raspberry Pi, mediante WiFi. L'ESP32 è configurato come MQTT Client con l'utilizzo del kernel FreeRTOS, il quale permette il coordinamento dei vari task. Il Raspberry Pi sarà configurato come server IoT, il quale svolgerà le funzioni di MQTT Broker, storage dati ed invio alert.

2 Strumenti e software utilizzati

Per lo sviluppo del progetto sono stati utilizzati i seguenti strumenti:

- **Raspberry Pi 4B:** computer a scheda singola basato sul sistema operativo Linux che, opportunamente configurato, permette l'utilizzo in vari campi dell'automazione. Si basa su un system-on-a-chip, ovvero è formato da un circuito integrato che in un solo chip contiene un intero sistema e utilizza una micro SD per il boot e per la memoria ROM non volatile. All'interno del Raspberry Pi è possibile installare, tramite apposito software, un qualsiasi sistema operativo.
- **BMP280:** sensore di temperatura e pressione.
- **BH1750:** sensore di luminosità.
- **Modulo RTC PCF8523:** è un real-time clock, ovvero un dispositivo con funzione di orologio.

Mentre i software utilizzati sono i seguenti:

- **Visual Studio Code:** editor di codice sorgente sviluppato da Microsoft per Windows, Linux e Mac OS. Permette, tramite apposita estensione, la programmazione di dispositivi remoti mediante connessione SSH.
- **FreeRTOS:** sistema operativo open source in tempo reale per microcontrollori e microprocessori. Verrà utilizzato nell'ESP32.
- **Mosquitto:** viene utilizzato come broker MQTT open-source all'interno del Raspberry Pi. Il broker MQTT è responsabile della ricezione e del filtraggio dei messaggi, dell'identificazione dei client iscritti e l'invio dei messaggi ad essi.
- **Raspberry Pi OS:** è il sistema operativo installato sul Raspberry Pi.
- **RealVNC Viewer:** è un software che permette l'accesso remoto ad un dispositivo.

3 Sviluppo Progetto

3.1 Collegamento dispositivi

I dispositivi per poter funzionare e lavorare tra loro necessitano di un cablaggio su una bread-board.

Sono stati collegati i seguenti componenti:

- **BMP280:** sensore di temperatura e pressione.
- **BH1750:** sensore di luminosità.
- **Modulo RTC PCF8523:** è un real-time clock, ovvero un dispositivo con funzione di orologio.
- **ESP32:** microcontrollore nel quale caricare il codice per gestire i vari task.

Il cablaggio è stato effettuato considerando lo schema seguente:

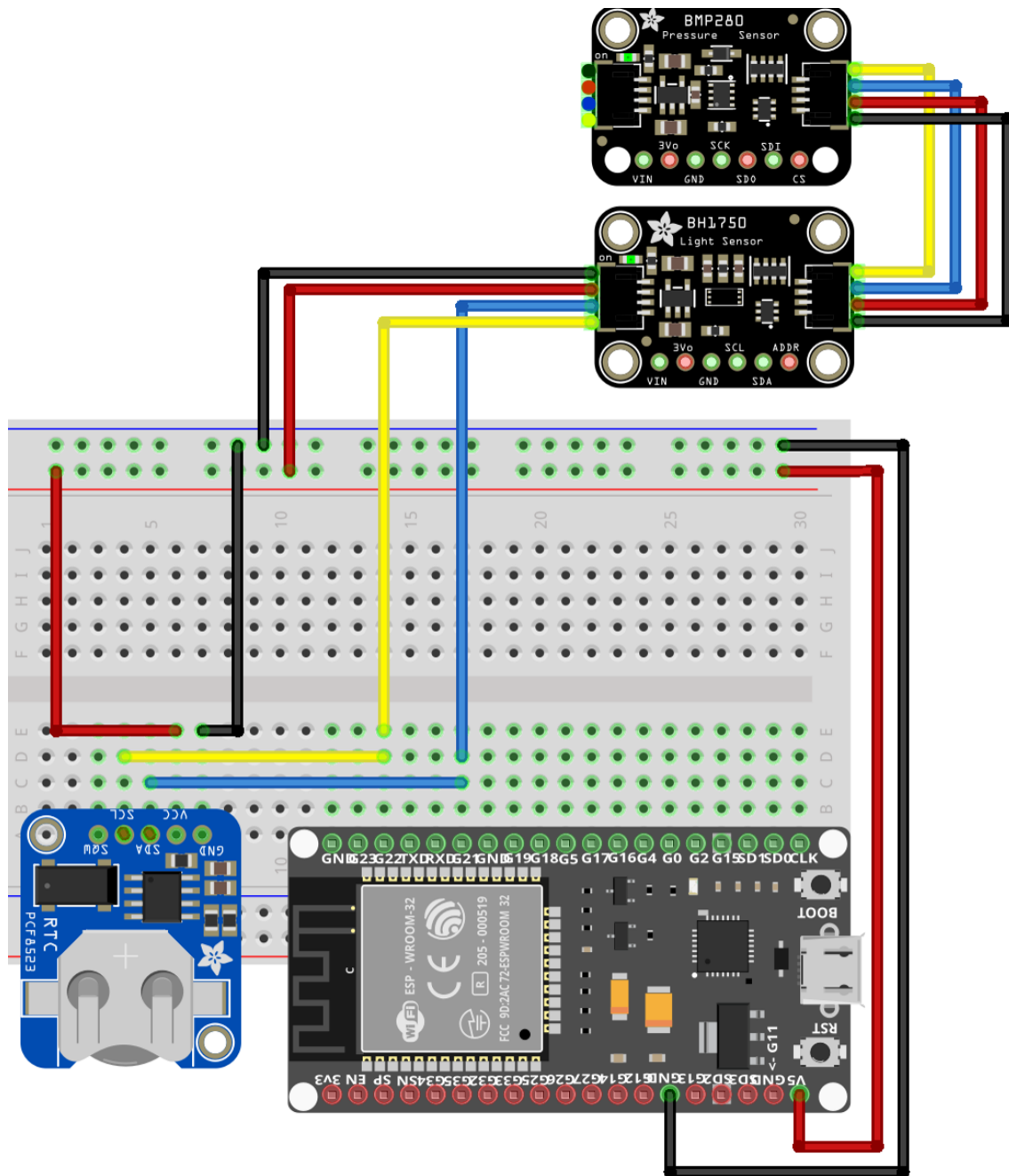


Figura 2: Schema di cablaggio del progetto

In seguito l'ESP32 è stato collegato al PC utilizzando un cavo Micro USB.

3.2 Configurazione Raspberry

Per poter utilizzare il Raspberry è stato necessario installare all'interno della sua scheda SD il sistema operativo dedicato, attraverso il software Raspberry Pi Imager. Nell'immagine sottostante si può vedere la schermata iniziale di Raspberry Pi Imager, dove si andrà a scegliere il modello del Raspberry, il sistema operativo da installare e la periferica su cui farlo; nel nostro caso, è stato scelto il modello Raspberry Pi 4B, il Raspberry Pi OS come sistema operativo e la scheda SD come periferica per installare l'OS. Inoltre, è possibile scegliere se pre-configurare il dispositivo. In tal caso, verrà richiesto di inserire nome utente, password, SSID e password

3 Sviluppo Progetto

del WiFi. Prima di salvare la configurazione è necessario abilitare l'SSH così da permettere la connessione da remoto.

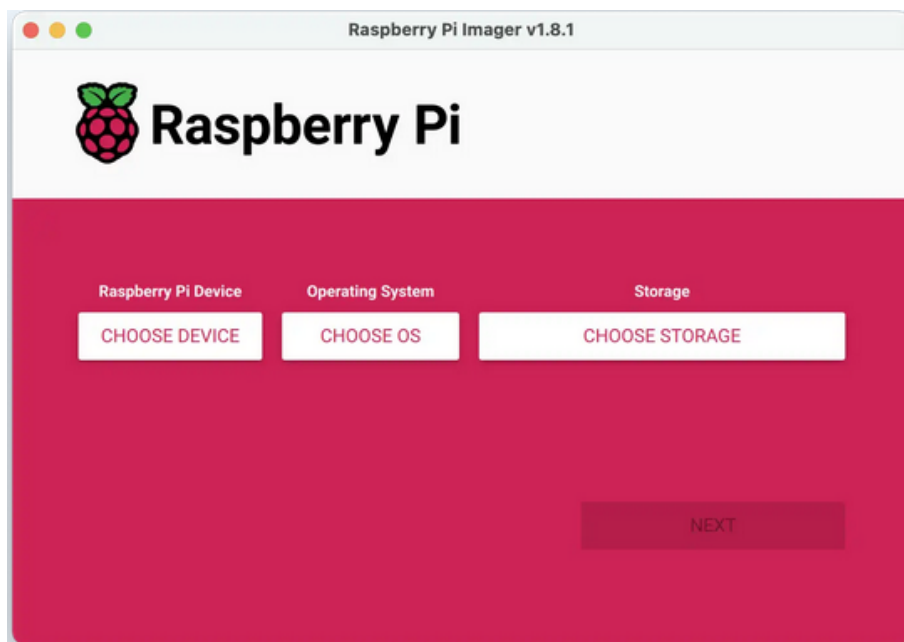


Figura 3: Raspberry Pi Imager

Dopodiché viene inserita la scheda SD sul Raspberry e per poter terminare la configurazione è necessario la prima volta collegare il dispositivo ad uno schermo, tramite cavo microHDMI, ad una tastiera ed un mouse.

Se non è stato pre-configurato, verranno impostate le credenziali (nome utente e password). Per la connessione Internet si può scegliere se collegare direttamente il cavo Ethernet o se collegarsi al WiFi.

Il protocollo **SSH** o Secure Shell permette una comunicazione client-server sicura tra dispositivi.

Dalle impostazioni del Raspberry, sulla sezione **Interfacce**, viene abilitato il servizio SSH, come da figura sottostante.

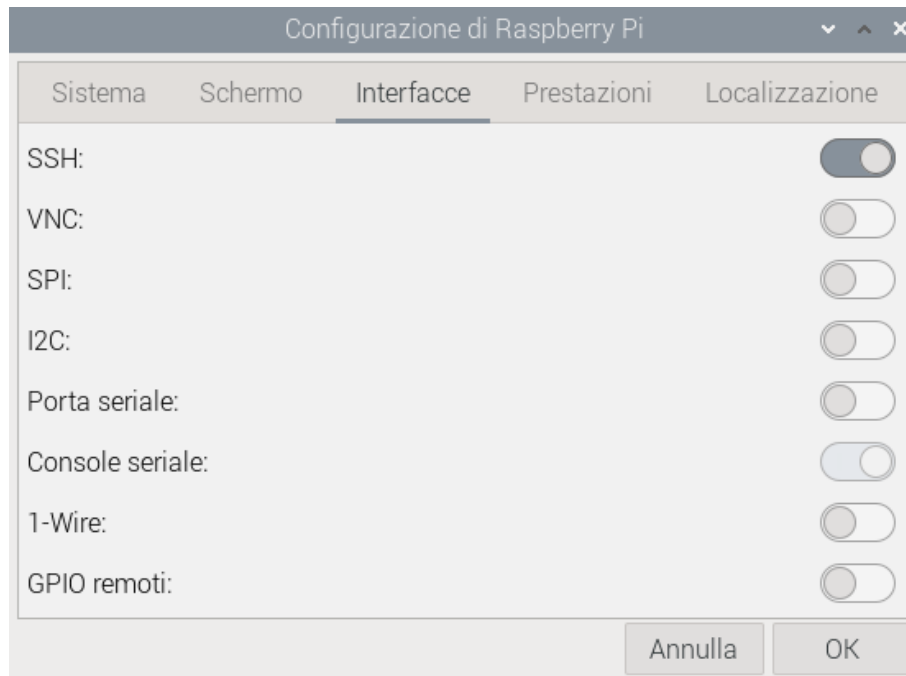


Figura 4: Abilitazione SSH

Ora, per configurare la connessione SSH è possibile utilizzare **PuTTY**, che è un client SSH gratuito portabile che permette la gestione dei dispositivi da remoto o il software **RealVNC Viewer**.

In entrambi i casi, i software devono essere installati nella macchina su cui si intende lavorare, nel nostro caso una macchina Windows. Se si utilizza PuTTY, si dovrà specificare sotto il campo **Host Name (or IP address)** il nome del Raspberry e selezionare come **Connection type** l'opzione **SSH**.

Dopo aver cliccato “**Open**”, verranno inserite le credenziali di Raspberry sulla finestra visualizzata.

In questo modo è stata instaurata una connessione SSH tra il PC ed il dispositivo.

Se invece si utilizza RealVNC Viewer si inserisce il nome del dispositivo Raspberry, che nel nostro caso è **raspberrypi.local** e si instaura la connessione.

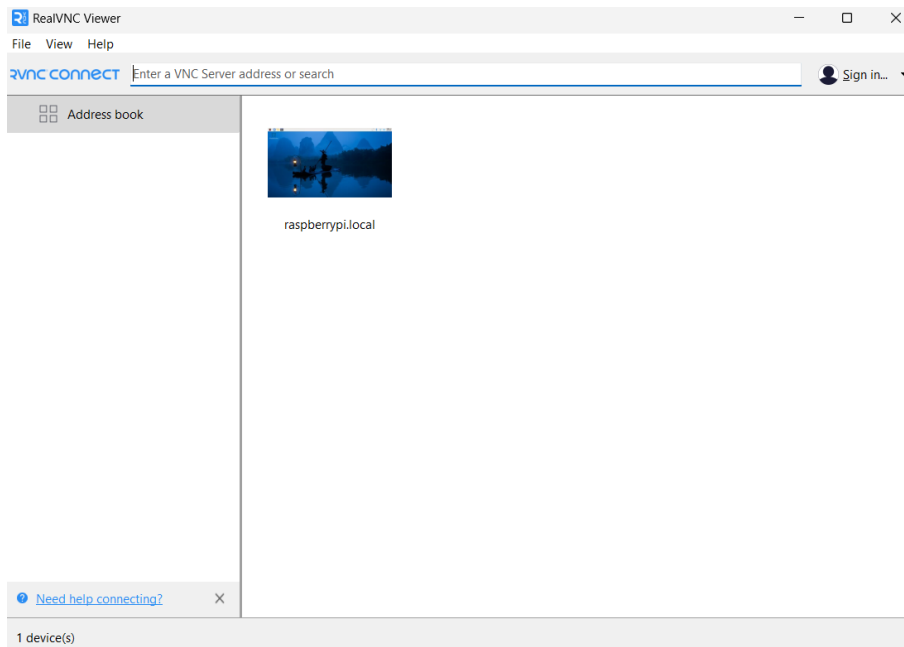


Figura 5: RealVNC Viewer

Per effettuare la programmazione del Raspberry direttamente dal PC, è necessario installare l'estensione **REMOTE-SSH** su Visual Studio Code, aggiungere una nuova macchina, cliccando l'icona *Remote Explorer* sulla sinistra e cliccare sul "+". A quel punto sulla barra di ricerca in alto si dovranno inserire uno alla volta i dati della connessione instaurata e scegliere la tipologia di server a cui ci si vuole collegare, nel nostro caso Linux:

- **nome connessione:** nome Raspberry Pi, nel nostro caso **raspberrypi.local**
- **host:** nome host (da terminale inviare comando **hostname**)
- **nome utente:** nel nostro caso **brugia**

Dopodiché per confermare, si dovrà inserire la password impostata nel Raspberry.

Essendo il Raspberry connesso alla rete ha un proprio indirizzo IP, che può essere statico o dinamico. Per assegnare al Raspberry un indirizzo IP statico, bisogna:

1. Inviare da terminale il seguente comando per modificare la connessione:

```
1 sudo nmtui edit "Nome connessione"
```

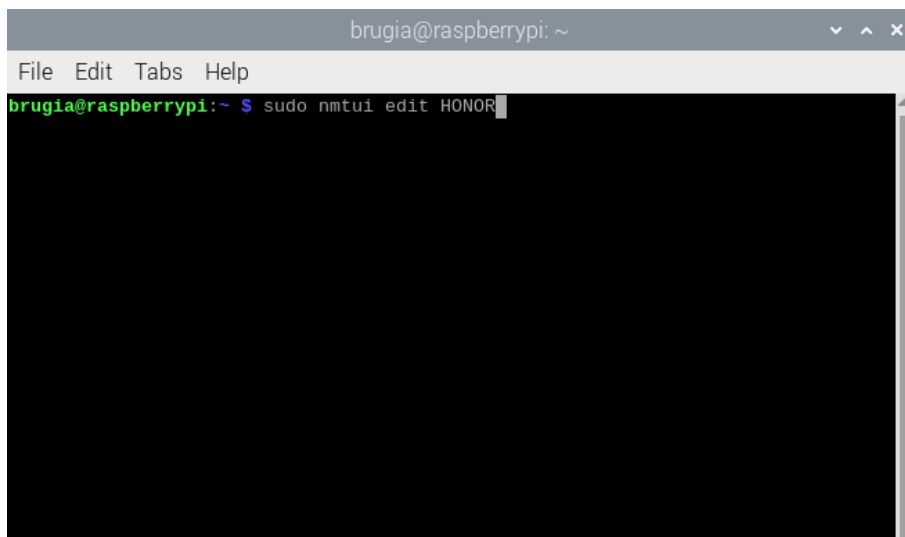


Figura 6: Modifica connessione

2. Nella sezione **CONFIGURAZIONE IPv4** dell'editor che si è aperto, cambiare l'opzione **Automatico** in **Manuale**
3. Cliccare su **Mostra** ed inserire sotto il campo **Indirizzi**, l'indirizzo IP statico da assegnare al dispositivo.
4. Inserire il Gateway
5. Inserire il DNS
6. Cliccare **OK** per salvare la configurazione.
7. Riavviare il servizio di rete, inviando da terminale:

```
sudo systemctl restart NetworkManager
```

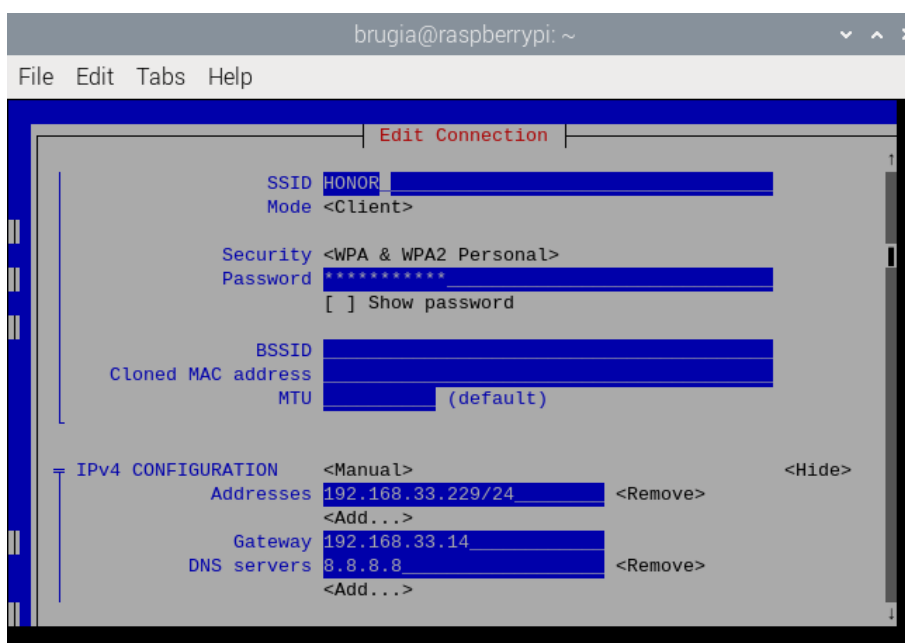


Figura 7: Configurazione IP statico

3 Sviluppo Progetto

Per verificare l'indirizzo IP assegnato al Raspberry si può trascinare il mouse sopra l'icona di rete oppure inviare da terminale il comando **hostname -I**.

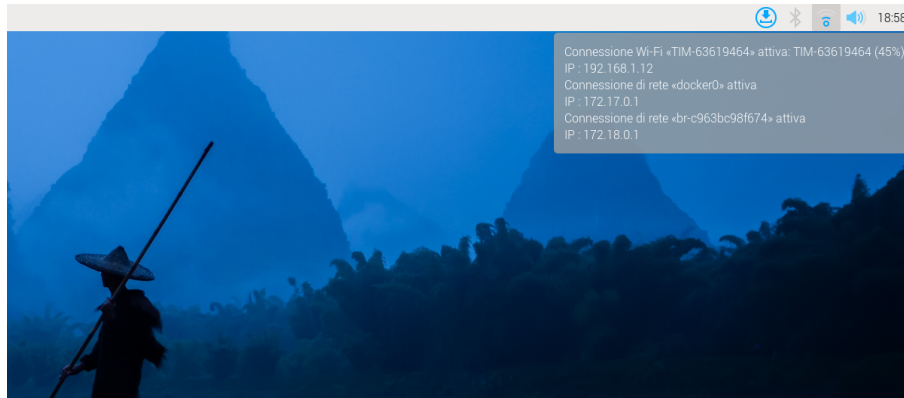


Figura 8: Visualizzazione IP

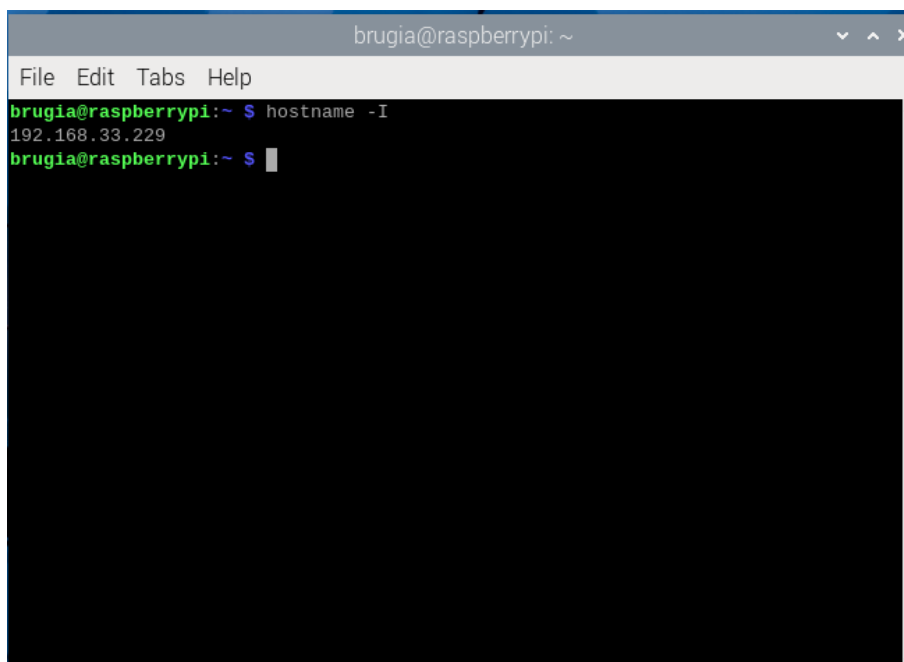


Figura 9: Visualizzazione IP da terminale

Il Raspberry Pi dovrà svolgere la funzione di server IoT, ovvero si occuperà di svolgere la funzione di MQTT Broker, storage dati ed invio alert.

L'**MQTT** è un protocollo di messaggistica standard per l'IoT, che, attraverso un broker, consente lo scambio di messaggi tra dispositivi.

Il broker MQTT è il sistema back-end, responsabile della ricezione e del filtraggio dei messaggi, della decisione di chi è interessato a loro e della pubblicazione del messaggio a tutti i client iscritti.

Il client MQTT è un qualsiasi dispositivo che utilizza il servizio MQTT.

Tra i vari broker disponibili, viene usato **Mosquitto**, che è il più comune, il quale è stato installato sul Raspberry tramite VS Code, con i seguenti passaggi:

3 Sviluppo Progetto

1. Viene aggiornato il sistema:

```
sudo apt update && sudo apt upgrade
```

2. Si installa Mosquitto :

```
sudo apt install -y mosquitto mosquitto-clients
```

3. Si lancia il comando per avviare Mosquitto automaticamente all'avvio:

```
sudo systemctl enable mosquitto.service
```

4. Si testa l'installazione:

```
mosquitto -v
```

Viene poi abilitato l'accesso remoto senza autenticazione:

1. Si apre il file **mosquitto.conf**

```
sudo nano /etc/mosquitto/mosquitto.conf
```

2. Si aggiungono alla fine del file le seguenti istruzioni per configurare l'accesso non autenticato:

```
listener 1883
allow_anonymous true
```

3. Si salva il file e si riavvia Mosquitto.

Per far avviare in automatico gli script Python è necessario creare due file **service** all'interno della cartella **systemd** del Raspberry. Ogni file è composto da tre sezioni:

1. **Unit:**

- **Description:** nome servizio
- **After:** determina quando inizia il servizio.

2. **Service:**

- **ExecStart:** contiene il percorso dello script che deve essere eseguito in automatico all'avvio.
- **Restart:** indica se il servizio deve essere riavviato in caso di uscita.
- **User:** identifica il nome del Raspberry.

3. **Install:**

3 Sviluppo Progetto

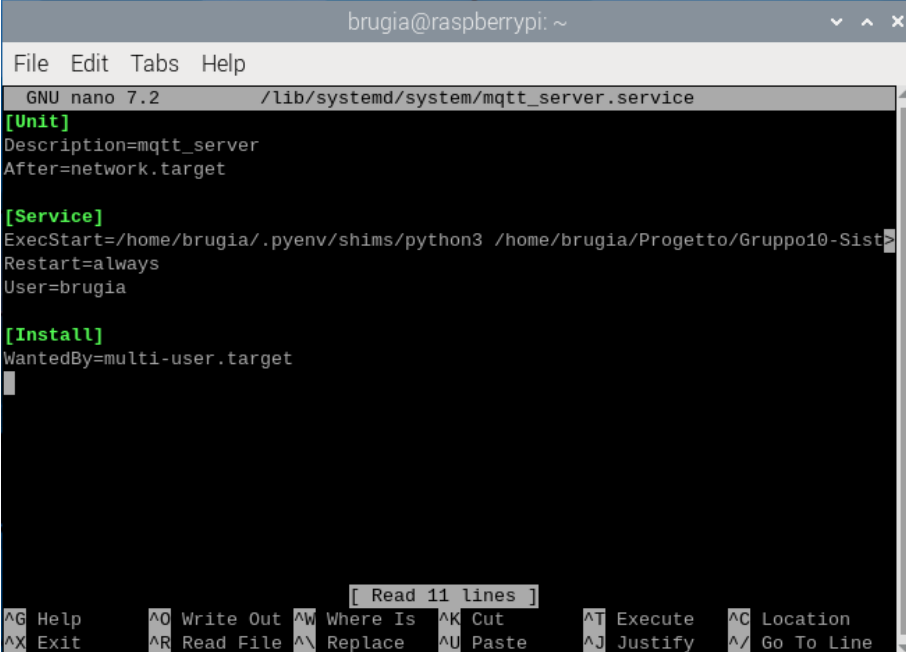
- **WantedBy**: specifica l'attivazione di un'unità. Impostandola su **multi-user.target**, si può stabilire una relazione di dipendenza che eseguirà il programma quando il controllo del sistema passa alla shell multiutente.

Nel nostro caso, viene fatto eseguire lo script che si occupa della ricezione dei dati dall'MQTT broker con il relativo caricamento in un database e lo script relativo alla gestione del bot Telegram.

Per creare i due file, da terminale si lanciano i seguenti comandi:

```
sudo nano /lib/systemd/system/mqtt_server.service
sudo nano /lib/systemd/system/telegram_bot.service
```

Nelle due figure sottostanti si possono vedere i file **service** creati con le relative informazioni.

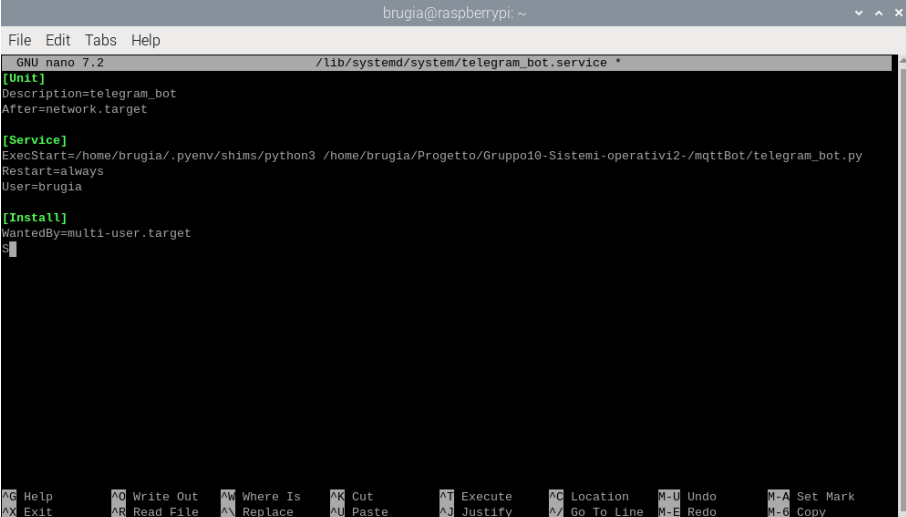


```
brugia@raspberrypi: ~
File Edit Tabs Help
GNU nano 7.2 /lib/systemd/system/mqtt_server.service
[Unit]
Description=mqtt_server
After=network.target

[Service]
ExecStart=/home/brugia/.pyenv/shims/python3 /home/brugia/Progetto/Gruppo10-Sist>
Restart=always
User=brugia

[Install]
WantedBy=multi-user.target
[ Read 11 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Figura 10: File Service MQTT



```
brugia@raspberrypi: ~
File Edit Tabs Help
GNU nano 7.2 /lib/systemd/system/telegram_bot.service *
[Unit]
Description=telegram_bot
After=network.target

[Service]
ExecStart=/home/brugia/.pyenv/shims/python3 /home/brugia/Progetto/Gruppo10-Sistemi-operativi2-/mqttBot/telegram_bot.py
Restart=always
User=brugia

[Install]
WantedBy=multi-user.target
S
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location  ^M Undo      ^I Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line ^- Redo      ^- Copy
```

Figura 11: File Service Bot

3 Sviluppo Progetto

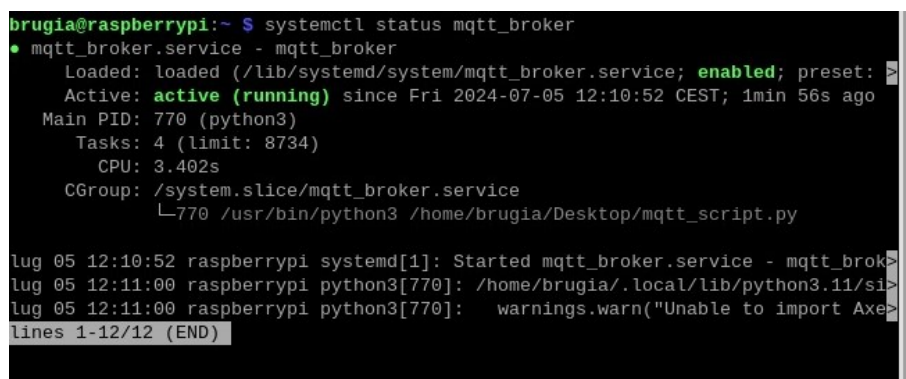
Dopo aver salvato i file, è necessario effettuare il riavvio del dispositivo. Alla riaccensione si dovrà abilitare ed avviare i due servizi con:

```
3 sudo systemctl enable mqtt_server.service
sudo systemctl start mqtt_server.service
sudo systemctl enable telegram_bot.service
sudo systemctl start telegram_bot.service
```

Per verificare che il servizio venga effettivamente eseguito in automatico, si invia da terminale il comando:

```
1 sudo systemctl status mqtt_server.service
sudo systemctl status telegram_bot.service
```

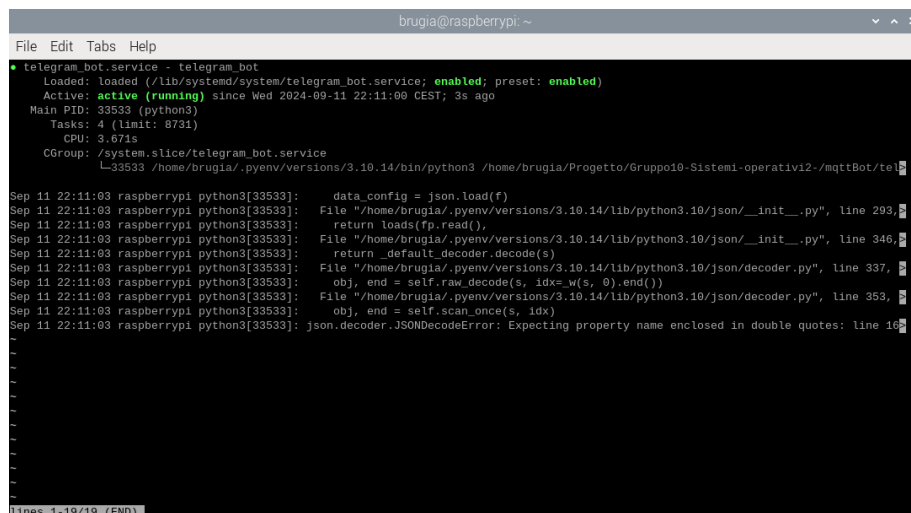
e si controlla che lo stato di **Active** sia **active (running)**, come da figura sottostante.



```
brugia@raspberrypi:~ $ systemctl status mqtt_broker
● mqtt_broker.service - mqtt_broker
   Loaded: loaded (/lib/systemd/system/mqtt_broker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2024-07-05 12:10:52 CEST; 1min 56s ago
     Main PID: 770 (python3)
        Tasks: 4 (limit: 8734)
            CPU: 3.402s
       CGroup: /system.slice/mqtt_broker.service
              └─770 /usr/bin/python3 /home/brugia/Desktop/mqtt_script.py

lug 05 12:10:52 raspberrypi systemd[1]: Started mqtt_broker.service - mqtt_broker.
lug 05 12:11:00 raspberrypi python3[770]: /home/brugia/.local/lib/python3.11/site-packages/axemqtt/
lug 05 12:11:00 raspberrypi python3[770]: warnings.warn("Unable to import Axemqtt", DeprecationWarning)
lines 1-12/12 (END)
```

Figura 12: Stato MQTT Broker Service



```
brugia@raspberrypi:~
File Edit Tabs Help
● telegram_bot.service - telegram_bot
   Loaded: loaded (/lib/systemd/system/telegram_bot.service; enabled; preset: enabled)
   Active: active (running) since Wed 2024-09-11 22:11:00 CEST; 3s ago
     Main PID: 33533 (python3)
        Tasks: 4 (limit: 8731)
            CPU: 3.671s
       CGroup: /system.slice/telegram_bot.service
              └─33533 /home/brugia/.pyenv/versions/3.10.14/bin/python3 /home/brugia/Progetto/Gruppo10-Sistemi-operativi2-/mqttBot/telegram_bot.py

Sep 11 22:11:03 raspberrypi python3[33533]: data_config = json.load(f)
Sep 11 22:11:03 raspberrypi python3[33533]: File "/home/brugia/.pyenv/versions/3.10.14/lib/python3.10/json/__init__.py", line 293, in load
Sep 11 22:11:03 raspberrypi python3[33533]: return loads(fp.read(),
Sep 11 22:11:03 raspberrypi python3[33533]: File "/home/brugia/.pyenv/versions/3.10.14/lib/python3.10/json/__init__.py", line 346, in loads
Sep 11 22:11:03 raspberrypi python3[33533]: return _default_decoder.decode(s)
Sep 11 22:11:03 raspberrypi python3[33533]: File "/home/brugia/.pyenv/versions/3.10.14/lib/python3.10/json/decoder.py", line 337, in decode
Sep 11 22:11:03 raspberrypi python3[33533]: obj, end = self.raw_decode(s, idx=_w(s, 0).end())
Sep 11 22:11:03 raspberrypi python3[33533]: File "/home/brugia/.pyenv/versions/3.10.14/lib/python3.10/json/decoder.py", line 353, in raw_decode
Sep 11 22:11:03 raspberrypi python3[33533]: obj, end = self.scan_once(s, idx)
Sep 11 22:11:03 raspberrypi python3[33533]: json.decoder.JSONDecodeError: Expecting property name enclosed in double quotes: line 16, column 1
lines 1-19/19 (END)
```

Figura 13: Stato Telegram Bot Service

3.3 Librerie

In questo progetto sono stati prodotti tre script:

- Script creato per programmare l'ESP32 e sviluppato con Arduino IDE

- Script Python per la ricezione dei messaggi dall'MQTT broker e caricamento dei dati ricevuti
- Script Python per la gestione del bot Telegram

Per quanto riguarda lo script sviluppato con Arduino, per prima cosa bisogna collegare l'ESP32 al PC tramite cavo USB, se il driver non viene installato automaticamente, è necessario scaricarlo dal sito ufficiale. Una volta creato lo script, si deve scegliere la porta e la tipologia di dispositivo. Nel nostro caso la porta è la *COM3* ed il dispositivo è *uPesy ESP32 Wroom DevKit*.

Per poter scaricare ed installare le librerie, bisogna cliccare sull'icona del *Gestore Librerie* a sinistra e ricercare la libreria di interesse.

Per scegliere la libreria è consigliato ricercarla prima nel sito ufficiale Arduino e poi selezionarla all'interno del Gestore Librerie.

Nel nostro caso, sono state installate ed importate le seguenti librerie:

1. **Adafruit Sensor:** libreria base necessaria per tutti i sensori
2. **Adafruit BMP280:** libreria per il sensore della temperatura e pressione BMP280
3. **BH1750:** libreria per il sensore di luminosità BH1750
4. **RTCLib:** libreria per il sensore RTC
5. **WiFi:** libreria per la connessione internet
6. **PubSubClient:** libreria per lo scambio dei messaggi MQTT
7. **NTPClient:** libreria utilizzata per creare il client NTP per la sincronizzazione della data e dell'ora
8. **WiFiUDP:** libreria per la gestione dei pacchetti

Invece per quanto riguarda gli script Python, per la gestione di diverse versioni di Python all'interno della stessa macchina è stato utilizzato Pyenv, il quale permette di installare diverse versioni di Python e relativi ambienti virtuali, mantenendole separate tra loro. Pyenv viene installato automaticamente lanciando da terminale:

```
curl https://pyenv.run | bash
```

Dopodiché per poter utilizzare Pyenv in tutti gli ambienti e quindi poterlo utilizzare da terminale, è necessario inviare da riga di comando i seguenti comandi:

```
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
echo 'command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(pyenv init -)"' >> ~/.bashrc
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.profile
echo 'command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.profile
echo 'eval "$(pyenv init -)"' >> ~/.profile
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bash_profile
echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bash_profile
echo 'eval "$(pyenv init -)"' >> ~/.bash_profile
```

3 Sviluppo Progetto

Ora, si passa all'installazione delle dipendenze necessarie alla compilazione delle diversioni di Python con il comando:

```
1 sudo apt update; sudo apt install build-essential libssl-dev zlib1g-dev \
  libbz2-dev libreadline-dev libsqlite3-dev curl git \
  libncursesw5-dev xz-utils tk-dev libxml2-dev libxmlsec1-dev libffi-dev \
  liblzma-dev
```

Per verificare che Pyenv sia stato installato correttamente si può inviare da terminale il comando:

```
which pyenv
```

e verificare che Pyenv sia nel nostro path. A questo punto, si può procedere con l'installazione della versione di Python necessaria mediante il comando:

```
pyenv install <versione>
```

Nel nostro caso si è installata la versione 3.10.15.

A questo punto, si passa all'installazione di Poetry, lanciando da terminale:

```
curl -sSL https://install.python-poetry.org | python3
```

e si va a modificare il valore della variabile PATH all'interno del file *bashrc* presente nella home con:

```
export PATH="$HOME/.local/bin:$PATH"
```

In questo modo si esporta la variabile PATH aggiornata con l'aggiunta del path

```
$HOME/.local/bin
```

all'inizio di suddetta variabile.

Se eseguendo

```
poetry --version
```

si ottiene la versione, significa che l'installazione è avvenuta con successo.

Al termine dello sviluppo del progetto è stato creato il file **requirements.txt**, attraverso il comando:

```
poetry export --without-hashes --format=requirements.txt > requirements.txt
```

il quale permette di installare velocemente tutte le librerie utilizzate nel progetto.

```
requirements.txt
1 anyio==4.4.0 ; python_version >= "3.10" and python_version < "4.0"
2 certifi==2024.8.30 ; python_version >= "3.10" and python_version < "4.0"
3 contourpy==1.3.0 ; python_version >= "3.10" and python_version < "4.0"
4 cycler==0.12.1 ; python_version >= "3.10" and python_version < "4.0"
5 exceptiongroup==1.2.2 ; python_version >= "3.10" and python_version < "3.11"
6 fonttools==4.53.1 ; python_version >= "3.10" and python_version < "4.0"
7 h11==0.14.0 ; python_version >= "3.10" and python_version < "4.0"
8 httpcore==1.0.5 ; python_version >= "3.10" and python_version < "4.0"
9 httpx==0.27.2 ; python_version >= "3.10" and python_version < "4.0"
10 idna==3.8 ; python_version >= "3.10" and python_version < "4.0"
11 kiwisolver==1.4.7 ; python_version >= "3.10" and python_version < "4.0"
12 matplotlib==3.9.2 ; python_version >= "3.10" and python_version < "4.0"
13 mysql-connector-python==9.0.0 ; python_version >= "3.10" and python_version < "4.0"
14 numpy==2.1.1 ; python_version >= "3.10" and python_version < "4.0"
15 packaging==24.1 ; python_version >= "3.10" and python_version < "4.0"
16 paho-mqtt==2.1.0 ; python_version >= "3.10" and python_version < "4.0"
17 pillow==10.4.0 ; python_version >= "3.10" and python_version < "4.0"
18 pyparsing==3.1.4 ; python_version >= "3.10" and python_version < "4.0"
19 python-dateutil==2.9.0.post0 ; python_version >= "3.10" and python_version < "4.0"
20 python-telegram-bot==21.5 ; python_version >= "3.10" and python_version < "4.0"
21 six==1.16.0 ; python_version >= "3.10" and python_version < "4.0"
22 sniffio==1.3.1 ; python_version >= "3.10" and python_version < "4.0"
23 typing-extensions==4.12.2 ; python_version >= "3.10" and python_version < "3.11"
```

Figura 14: File requirements.txt

3.4 FreeRTOS

FreeRTOS è un sistema operativo real-time in grado di gestire più attività in parallelo e di rispettare i tempi di esecuzione dei processi.

Per fare ciò vengono utilizzati i **task**, ovvero delle funzioni contenenti una serie di istruzioni da eseguire in loop.

Per lo svolgimento del progetto, sono stati creati ed implementati i seguenti task:

- **sensorTask**: viene usato per leggere i dati dai sensori BMP280 e BH1750, quali temperatura, pressione e luminosità.
- **mqttTask**: viene usato per instaurare la connessione MQTT.
- **syncTimeTask**: viene usato per sincronizzare i dati del sensore RTC, quali data e ora con quelli del server NTP.

Per creare i task vengono utilizzate le seguenti funzioni definite all'interno del **setup**:

```
xTaskCreatePinnedToCore(sensorTask, "SensorTask", 4096, NULL, 1, &
    sensorTaskHandle, 0);
xTaskCreatePinnedToCore(mqttTask, "MQTTTask", 8192, NULL, 1, &
    mqttTaskHandle, 1);
xTaskCreatePinnedToCore(syncTimeTask, "SyncTimeTask", 4096, NULL, 1, &
    syncTimeTaskHandle, 1);
```

Listing 1: Creazione Task

che prendono come parametri:

- **Puntatore alla funzione** – il nome del task
- **Descrizione del task**

- **Dimensione stack**
- **Parametri stack**
- **Priorità:** possono essere assegnati valori da 0 a (`configMAX_PRIORITIES - 1`). Quest'ultimo viene definito all'interno del file di configurazione **FreeRTOSConfig.h**. Per task che hanno pari priorità, come nel nostro caso, viene assegnato valore 1. Valori bassi di priorità indicano attività a bassa priorità.
- Possibilità di accedere al task dall'esterno
- I valori 0 o 1 indicano il numero di indice della CPU a cui l'attività deve essere assegnata

Dopodiché vengono definiti i task creati, inserendo le istruzioni che dovranno essere svolte in loop.

Le istruzioni definite al loro interno, come per la lettura dei valori rilevati dai sensori, per la pubblicazione dei dati su MQTT e per il controllo delle soglie, verranno descritte in dettaglio successivamente.

All'interno del setup, prima della creazione dei task, come da codice sottostante, viene effettuata la connessione al WiFi, richiamando l'apposita funzione `connectToWiFi()` e vengono effettuati dei controlli iniziali per verificare la rilevazione e l'effettivo funzionamento e collegamento dei dispositivi utilizzati, compresa la sincronizzazione della data e dell'ora.

Per essere sicuri che la data e l'ora rilevata sia quella reale è stata inclusa la libreria `NTPClient` che consente la sincronizzazione della data e dell'ora da un server NTP.

Nel caso in cui non venga rilevato il sensore, viene visualizzato in output un messaggio di errore.

```
void setup() {  
  Serial.begin(115200);  
  connectToWiFi();  
  client.setServer(mqttBroker, mqttPort);  
  
  if (!bmp.begin()) {  
    Serial.println("Impossibile trovare un sensore valido BMP280, controlla  
    il cablaggio!");  
    while (1);  
  }  
  
  if (!lightSensor.begin()) {  
    Serial.println("Impossibile trovare un sensore valido BH1750, controlla  
    il cablaggio!");  
    while (1);  
  }  
  
  if (!rtc.begin()) {  
    Serial.println("Impossibile trovare un sensore valido RTC, controlla il  
    cablaggio!");  
    while (1);  
  }  
  
  syncTimeWithNTP();  
}
```

Listing 2: Rilevazione sensori e sincronizzazione data e ora

Ogni sensore viene però prima inizializzato richiamando l'apposita libreria:

3 Sviluppo Progetto

```
Adafruit_BMP280 bmp;  
2 BH1750 lightSensor;  
RTC_PCF8523 rtc;
```

Listing 3: Inizializzazione sensori

Viene definito un NTP Client per richiedere la data e l'ora ad un server NTP, così da avere sempre data e ora aggiornate. Sono stati specificati anche l'offset e l'intervallo di aggiornamento, espresso in millisecondi.

```
WiFiUDP ntpUDP;  
NTPClient timeClient(ntpUDP, "pool.ntp.org", 3600*2, 60000);
```

Listing 4: NTP Client

Viene poi inizializzato l'NTP Client all'interno della funzione per la connessione al WiFi `timeClient.begin()`.

Dopodiché nel setup viene richiamata la funzione apposita `syncTimeWithNTP()` per la sincronizzazione della data e dell'ora.

Nel codice sottostante, si può vedere in dettaglio come avviene la connessione al WiFi tramite `connectToWiFi()`.

La connessione alla rete si effettua attraverso la funzione `Wifi.begin(ssid, password)` alla quale viene passato l'SSID e la password del WiFi. Finché lo stato della connessione non risulta essere connesso, si ritenta la connessione.

```
void connectToWiFi() {  
    Serial.println("Connessione al WiFi..");  
    Wifi.begin(ssid, password);  
    while (Wifi.status() != WL_CONNECTED) {  
5        delay(1000);  
        Serial.println("Connettendosi al WiFi...");  
    }  
    Serial.println("Connesso al WiFi");  
    timeClient.begin();  
10 }
```

Listing 5: Connessione WiFi

Al fine di far comunicare i vari dispositivi, vengono utilizzate due librerie:

1. **WiFi:** per consentire la connessione alla rete e permette di definire server, client e inviare e ricevere messaggi.
2. **PubSubClient:** per scambiare messaggi MQTT.

Attraverso **WiFiClient** viene definito un nuovo client **espClient** che può connettersi alla rete e scambiare messaggi MQTT, attraverso `PubSubClient client(espClient)`.

Per semplificare la gestione del codice, vengono definite all'inizio le costanti e le variabili globali relative ai task, al WiFi, alla porta, all'indirizzo IP e ai topic MQTT.

```
TaskHandle_t sensorTaskHandle, mqttTaskHandle, syncTimeTaskHandle;  
QueueHandle_t dataQueue;  
WiFiClient espClient;  
PubSubClient client(espClient);
```

```
5 const char *ssid = "HONOR";  
const char *password = "k9sD4@P4Re9";  
const char *mqttBroker = "192.168.33.229";  
const int mqttPort = 1883;
```

Listing 6: Dichiarazione costanti e variabili globali

3.5 ESP32, MQTT Broker e MQTT Client

L'acquisizione delle grandezze fisiche richieste avviene mediante i sensori collegati alla ESP32 che funge da nodo IoT. Questo microcontrollore, grazie alla connessione WiFi integrata e al protocollo MQTT, invierà i dati acquisiti al broker.

Come descritto precedentemente, il Raspberry assume il compito di MQTT Broker, mentre l'ESP32 funge da MQTT Client.

Il funzionamento di MQTT è il seguente:

- Il client MQTT stabilisce una connessione con il broker MQTT
- Una volta connesso, il client invia i messaggi al broker
- Quando il broker riceve il messaggio, lo inoltra ai dispositivi interessati

Per fare ciò, prima di tutto, è stata inclusa la libreria **PubSubClient**, appositamente per la messaggistica MQTT, che permette di ricevere e inviare messaggi.

All'interno del **setup** viene effettuata la connessione all'MQTT Broker, attraverso la funzione `client.setServer(mqttBroker, mqttPort)`, a cui viene passato l'indirizzo dell'MQTT Broker e la sua porta di default.

```
void setup() {  
  Serial.begin(115200);  
  connectToWiFi();  
  client.setServer(mqttBroker, mqttPort);  
}
```

Listing 7: Connessione MQTT Broker

Per la comunicazione e lo scambio dei dati, è necessario che la connessione con l'MQTT Broker sia sempre attiva, per questo è stato inserito all'interno del **void loop**, il controllo sullo stato della connessione.

Nel caso in cui, lo stato sia disconnesso, si tenta la connessione all'MQTT Broker, attraverso l'apposita funzione `client.connect()` ogni 5 secondi, visualizzando in output l'esito della connessione.

```
void loop() {  
  if (!client.connected()) {  
    connectToMQTT();  
  }  
5  client.loop();  
  
  delay(10000); // Attendi 10 secondi prima di controllare di nuovo  
}
```

```
10 void connectToWiFi() {
    Serial.println("Connessione al WiFi..");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
15         delay(1000);
        Serial.println("Connettendosi al WiFi...");
    }
    Serial.println("Connesso al WiFi");
    timeClient.begin();
20 }

void connectToMQTT() {
    Serial.println("Connessione all MQTT");
    while (!client.connected()) {
25         if (client.connect("ESP32Client")) {
            Serial.println("Connesso all MQTT Broker");
        } else {
            Serial.print("Fallita la connessione all MQTT, rc=");
            Serial.print(client.state());
30             Serial.println(" Riprova in 5 secondi");
            delay(5000);
        }
    }
}
```

Listing 8: Connessione WiFi e MQTT Broker

L'ESP32 si connette ad un server MQTT, avvia la connessione e attende di ricevere comandi da parte di un client. In questo caso i comandi arriveranno dal Raspberry Pi e dallo smartphone dell'utente iscritto al bot Telegram.

3.6 Sensori

L'ESP32 dovrà inviare costantemente i valori misurati dai sensori all'MQTT Broker, i quali verranno poi memorizzati all'interno di un database apposito. Nel caso in cui le misure rilevate superino una certa soglia, verrà generato e inviato un alert all'utente.

Per fare ciò è necessario sottoscrivere il client MQTT ai topic relativi alle misure rilevate dai vari sensori, nel **main** attraverso `client.subscribe("sensori")`.

```
client.subscribe("sensori")
print("Sottoscritto al topic: sensori")
```

Listing 9: Sottoscrizione topic

Ora, vediamo in dettaglio come sono stati sviluppati i vari task.

3.6.1 Sensor Task

Il BMP280 è un sensore utilizzato per misurare temperatura e pressione dell'ambiente circostante, mentre il BH1750 è un sensore in grado di rilevare la luminosità dell'ambiente.

3 Sviluppo Progetto

Per poter gestire la lettura e la memorizzazione dei dati dai sensori, viene definita una struct con tre variabili float rispettivamente per temperatura, pressione e luminosità e due array per memorizzare data e ora.

```
struct SensorData {  
    float temperature;  
    float pressure;  
    float light;  
5   char date[11];  
    char time[9];  
};
```

Listing 10: Struct SensorData

La rilevazione dei dati dai sensori avviene mediante un unico task **sensorTask()** e deve essere costante e ciclica, per questo sono state inserite le apposite funzioni di lettura all'interno di un ciclo while(). La lettura dei dati avviene mediante le apposite funzioni:

- **bmp.readTemperature()**: per la temperatura
- **bmp.readPressure() / 100.0**: per la pressione
- **lightSensor.readLightLevel()**: per la luminosità
- **rtc.now()**: per la data e ora, che vengono poi divise in anno, mese e giorno e ora, minuti e secondi.

```
void sensorTask(void *parameter) {  
    (void)parameter;  
    TickType_t xLastWakeTime = xTaskGetTickCount();  
4  
    while (1) {  
        // Leggi dati dai sensori  
        SensorData data;  
        data.temperature = bmp.readTemperature();  
9        data.pressure = bmp.readPressure() / 100.0; // Pressione in hPa  
        data.light = lightSensor.readLightLevel();  
  
        // Leggi l'ora corrente dal RTC  
        DateTime now = rtc.now();  
14        snprintf(data.date, sizeof(data.date), "%04d/%02d/%02d", now.year(), now.  
            .month(), now.day());  
        snprintf(data.time, sizeof(data.time), "%02d:%02d:%02d", now.hour(), now.  
            .minute(), now.second());  
  
        // Invia i dati alla coda  
19        if (xQueueSend(dataQueue, &data, portMAX_DELAY) != pdPASS) {  
            Serial.println("Coda piena, impossibile inviare i dati dei sensori");  
        }  
  
        vTaskDelayUntil(&xLastWakeTime, 1000 / portTICK_PERIOD_MS); // Ritardo  
        di 1 secondo  
24    }  
}
```

Listing 11: Task Lettura dati

3 Sviluppo Progetto

Per poter inviare i dati all'MQTT Broker viene creata una coda

```
dataQueue = xQueueCreate(10, sizeof(SensorData));
```

in grado di contenere 10 puntatori alla struttura SensorData.

In particolare, una volta rilevati i dati dai sensori e memorizzati all'interno della struct, i dati vengono inviati alla coda tramite la funzione xQueueSend().

```
    if (xQueueSend(dataQueue, &data, portMAX_DELAY) != pdPASS) {  
        Serial.println("Coda piena, impossibile inviare i dati dei sensori");  
    }  
  
5    vTaskDelayUntil(&xLastWakeTime, 1000 / portTICK_PERIOD_MS); // Ritardo  
    di 1 secondo  
    }  
}
```

Listing 12: Invio dati alla coda

3.6.2 MQTT Task

Il secondo task implementato è quello relativo all'MQTT, il quale ciclicamente riceve i dati dalla coda tramite la funzione xQueueReceive() e crea una stringa formattata con i dati dei sensori separati dalle virgole. Dopodiché l'ESP32 si occupa dell'invio dei dati all'MQTT Broker, visualizzando in output l'esito.

```
void mqttTask(void *parameter) {  
    (void)parameter;  
    SensorData data;  
  
5    while (1) {  
        // Ricevi i dati dalla coda  
        if (xQueueReceive(dataQueue, &data, portMAX_DELAY) == pdPASS) {  
            // Crea una stringa formattata con i dati dei sensori separati da  
            virgole  
            char payload[100];  
10           snprintf(payload, sizeof(payload), "%.2f,%.2f,%.2f,%s,%s", data.  
                temperature, data.pressure, data.light, data.date, data.time);  
  
            // Pubblica i dati su MQTT  
            if (client.publish("sensori", payload)) {  
                Serial.print("Dati inviati al Broker MQTT: ");  
15                Serial.println(payload);  
            } else {  
                Serial.println("Fallito nell'inviare i dati al Broker MQTT");  
            }  
  
20        }  
    }  
}
```

Listing 13: Ricezione e pubblicazione dati

3.6.3 SyncTimeTask

L'ultimo task creato viene impiegato per mantenere la data e l'ora dell'RTC aggiornata.

3 Sviluppo Progetto

Ogni ora viene prevista una sincronizzazione richiamando la funzione `syncTimeWithNTP()` che, dopo aver richiesto la data e l'ora al server NTP, le sovrascrive all'interno dell'RTC. Per fare ciò vengono impiegate due funzioni:

- `getEpochTime()`: viene restituisce il tempo trascorso dall'anno in cui fu introdotto il GMT
- `rtc.adjust(DateTime(epochTime))`: sovrascrive la data dell'RTC con data e ora rilevata dal server NTP.

```
void syncTimeTask(void *parameter) {  
    (void)parameter;  
    while (1) {  
        syncTimeWithNTP();  
5      vTaskDelay(3600000 / portTICK_PERIOD_MS); // Sincronizza ogni ora  
    }  
}
```

Listing 14: Task sincronizzazione data e ora

```
void syncTimeWithNTP() {  
    while (!timeClient.update()) {  
        timeClient.forceUpdate();  
4    }  
}
```

Listing 15: Sincronizzazione NTP

3.7 Bot Telegram

Telegram è un servizio di messaggistica che permette la creazione di bot. Di fatti, il bot Telegram permette di far interagire il client attraverso messaggi Telegram, in particolare permetterà all'utente di visualizzare i valori misurati, gli eventuali alert ed i grafici sugli andamenti delle grandezze, in base ad un particolare range.

Per creare un nuovo bot, viene utilizzato BotFather, che è il bot padre da cui si possono creare altri bot. Si cerca su Telegram **@BotFather** e si invia sulla chat il comando `/newbot`. Dopodiché il BotFather chiederà il nome e l'identificativo da associare al nuovo bot, per poi fornire il token identificativo, da salvare perché necessario per poter utilizzare le API Telegram e per la programmazione del bot. Più utenti possono iscriversi al Bot Telegram, inserendo il nickname della chat.

Una volta generato il bot, si passa all'implementazione del codice. Nel nostro caso, il bot viene configurato sia per l'invio che per la ricezione dei messaggi.

L'utente potrà inviare sulla chat i seguenti comandi:

1. `/start`: per avviare la comunicazione ed il bot invierà una risposta preimpostata da noi.
2. `/leggi`: per leggere i dati memorizzati all'interno del database relativi ad un range di tempo.

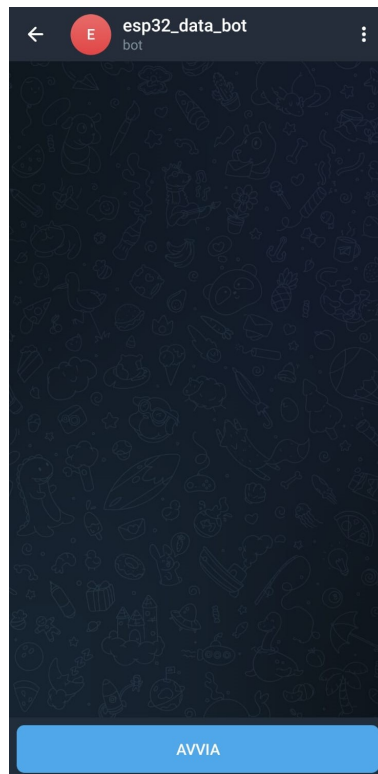


Figura 15: Bot Telegram

Prima di spiegare in dettaglio lo sviluppo del bot, nel paragrafo successivo viene illustrato come creare il database.

3.8 Creazione Database e caricamento dati

Il database utilizzato per questo progetto è un database **MySQL**, gestito con **phpMyAdmin** e viene eseguito su una macchina Windows locale ed un **server Apache**. Per poter configurare e gestire facilmente il DB, è stato installato **XAMPP** e creato il DB tramite l'interfaccia utente di phpMyAdmin.

XAMPP è un software che permette di testare il database in locale, phpMyAdmin è un software che permette la gestione di MySQL.

Per prima cosa bisogna eseguire XAMPP come amministratore, dopodiché nella schermata che si aprirà, verranno mostrati i vari servizi disponibili; nel nostro caso vengono avviati Apache e MySQL, cliccando sul pulsante Start. Cliccando su **Admin** in corrispondenza di MySQL, si aprirà la pagina di **phpMyAdmin**.

Per creare il database, si clicca su **create database** e a seguire su **create table** per generare la tabella.

Per questo progetto è stato creato un unico database **esp32_data** con all'interno due tabelle:

1. **sensor_data**: contenente tutti i dati relativi ai sensori
2. **chatids**: contenente tutti i chat ID degli utenti che si iscrivono alla chat del bot.

3 Sviluppo Progetto

Gli attributi della tabella `sensor_data` sono stati scelti in maniera tale da avere attributi di tipo:

- **float:** per temperatura, pressione e luminosità
- **int autoincrementale:** per gli ID (per selezionare questa tipologia, mettere la spunta sotto AI)
- **DATETIME:** per data e ora del sensore RTC

Gli attributi della tabella `chat_id` sono stati scelti in maniera tale da avere attributi di tipo:

- **int autoincrementale:** per gli ID
- **stringa:** per i chat id e per i nomi associati

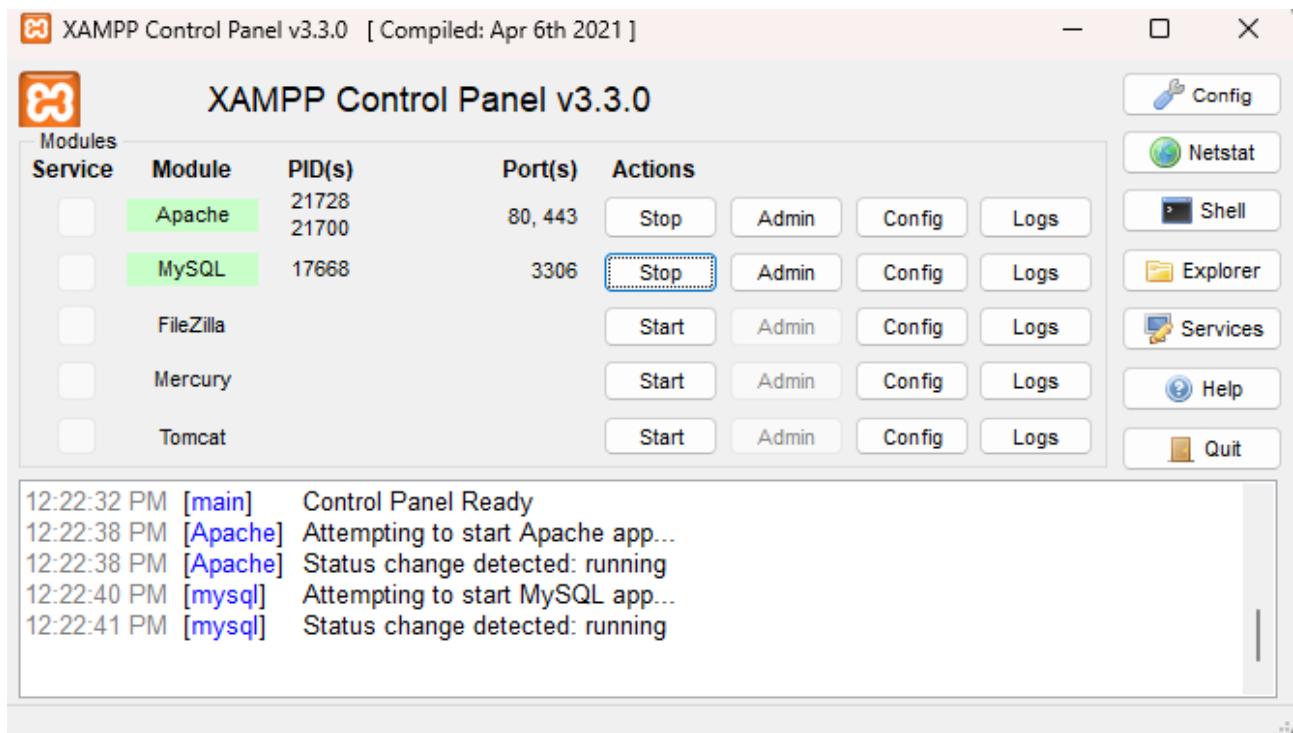


Figura 16: GUI Xampp

Una volta creato il database, si effettua la connessione ad esso, tramite apposita funzione **mysql.connector.connect** a cui vengono passati i parametri di configurazione, che vengono letti dal file JSON:

- L'indirizzo del server MySQL
- Il nome utente associato al database
- La password del database
- Il nome del database

```
main_dir = os.path.dirname(__file__)
config_path = os.path.join(main_dir, "Configurazione.json")

5   with open(config_path) as f:
        data_config = json.load(f)

    db_config = {
        "host": data_config["db_config"]["host"], # Indirizzo del server
        "user": data_config["db_config"]["user"], # Nome utente del
        "password": data_config["db_config"]["password"], # Password del
        "database": data_config["db_config"]["database"], # Nome del
    }
    database MySQL
```

Listing 16: Dati configurazione database

```
2   conn = mysql.connector.connect(**db_config)
    cursor = conn.cursor()
    print("Connessione al database MySQL avvenuta con successo")
```

Listing 17: Connessione al database

3.8.1 Sviluppo Script

Come anticipato sono stati creati due script, uno per la comunicazione con l'MQTT e l'altro per la gestione del bot. Al fine di ottimizzare il codice, lo script `telegram_bot.py` viene importato all'interno dello script `mqtt_server.py`: `import telegram_bot` Per comodità e facilità di gestione, è stato creato un file JSON contenente tutte le informazioni di configurazione, quali indirizzo IP del broker, i dati di accesso al database, il TOKEN del bot e le variabili delle grandezze misurate, quali soglia e isteresi.

```
{
    "broker_address" : "192.168.33.229",
    "db_config":{
        "host":"192.168.33.51",
        "user":"brugia",
        "password":"halo3000",
        "database":"esp32_data"
    },
    "temperatureThresholdHigh" : 5.5,
    "temperatureHysteresis" : 1.0,
    "lightThresholdHigh": 530.0,
    "lightHysteresis": 200.0,
    "pressureThresholdHigh": 1022.0,
    "pressureHysteresis" : 200.0,
    "TELEGRAM_TOKEN" : "6728633709:AAFXXIkfqvrAS2ub1CwPKIJ5PIdrKqdgEps"
}
```

Figura 17: File JSON

Il file JSON verrà letto tramite:

```
main_dir = os.path.dirname(__file__)
config_path = os.path.join(main_dir, "Configurazione.json")

with open(config_path) as f:
    data_config = json.load(f)
```

Listing 18: Lettura file JSON

in cui è stato specificato il percorso del file.

Il primo script, `mqtt_server.py`, si occupa della ricezione dei messaggi dal broker MQTT e del caricamento di essi all'interno del database `esp32_data` - tabella `sensor_data`.

Per fare ciò, prima di tutto, viene letto il file JSON, vengono associati i valori relativi alle grandezze misurate a delle variabili locali e viene effettuata la connessione al database.

```
if __name__ == "__main__":

    main_dir = os.path.dirname(__file__)
    config_path = os.path.join(main_dir, "Configurazione.json")

    with open(config_path) as f:
        data_config = json.load(f)

    db_config = {
        "host": data_config["db_config"]["host"], # Indirizzo del server
        "user": data_config["db_config"]["user"], # Nome utente del
        "password": data_config["db_config"]["password"], # Password del
        "database": data_config["db_config"]["database"] # Database MySQL
    }
```

```
    "password": data_config["db_config"]["password"], # Password del
database MySQL
    "database": data_config["db_config"]["database"], # Nome del
database MySQL
}

15
    temperatureThresholdEnabled = True
    temperatureActiveTh = data_config["temperatureThresholdHigh"]
    temperatureResetTh = temperatureActiveTh - data_config["
temperatureHysteresis"]

20
    lightThresholdEnabled = True
    lightActiveTh = data_config["lightThresholdHigh"]
    lightResetTh = lightActiveTh - data_config["lightHysteresis"]

    pressureThresholdEnabled = True
25
    pressureActiveTh = data_config["pressureThresholdHigh"]
    pressureResetTh = pressureActiveTh - data_config["pressureHysteresis"]

    topics = ["sensori"] # Un unico topic per i dati strutturati
# Connessione al database MySQL
30
    conn = mysql.connector.connect(**db_config)
    cursor = conn.cursor()
    print("Connessione al database MySQL avvenuta con successo")

    main()
```

Listing 19: Lettura file JSON e connessione database

Dopodiché all'interno del **main()** si crea un nuovo client MQTT, si imposta la funzione **on_message()** come callback per gestire i messaggi MQTT ricevuti, si connette il client al broker, lo si iscrive ai topics definiti all'interno dell'array:

```
    topics = ["sensori"]

def main():
    print("Esecuzione main...")

4
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

    client.on_message = on_message

    client.connect(data_config["broker_address"])
9
    print("Connessione al broker MQTT avvenuta con successo")

    # Sottoscrizione ai topic
    client.subscribe("sensori")
    print("Sottoscritto al topic: sensori")

14
    client.loop_forever()
```

Listing 20: MQTT Client

Nel dettaglio, la funzione **on_message()** decodifica il payload del messaggio separato dalle virgole e memorizza i primi 3 valori all'interno di tre variabili float, rispettivamente **temperatura**, **pressione** e **luminosità** ed i restanti due all'interno di due array rispettivamente **data_value** e **ora_value**.

Dopodiché si prepara la query con i valori rilevati della temperatura, pressione, luminosità, data e ora e si esegue l'inserimento all'interno del database, tramite **cursor.execute()**.

```
def on_message(client, userdata, message):
    topic = message.topic
    payload = message.payload.decode("utf-8")
    print(f"Ricevuto il messaggio '{payload}' nel topic '{topic}'")

    global temperatureThresholdEnabled, pressureThresholdEnabled,
    lightThresholdEnabled

    # Decodifica del payload separato da virgole
    try:
        data = payload.split(",")
        if len(data) == 5:
            temperatura = float(data[0])
            pressione = float(data[1])
            luminosita = float(data[2])
            data_value = data[3]
            ora_value = data[4]

            query = "INSERT INTO sensor_data (temperatura, pressione,
luminosita, data, ora) VALUES (%s, %s, %s, %s, %s)"
            cursor.execute(
                query, (temperatura, pressione, luminosita, data_value,
ora_value)
            )
            conn.commit()
            print(f"Dato inserito correttamente: {payload}")
        else:
            print(f"Payload incompleto: {payload}")
    except ValueError as e:
        print(f"Errore nel decodificare il payload: {e}")
```

Listing 21: Funzione ricezione messaggi MQTT

Nel momento in cui i dati vengono decodificati, viene effettuato il controllo di essi.

Come riportato nel file JSON, sono stati impostati i valori di soglia e di isteresi per ogni grandezza, tali per cui quando il valore rilevato dai sensori risulti essere superiore alla soglia, venga inviato un alert a tutti gli utenti iscritti al bot e venga disattivata la variabile di monitoraggio, così da evitare che arrivino all'utente continui messaggi. Il monitoraggio viene riattivato solo quando i valori misurati tornano ad essere inferiori o uguali al valore di soglia-valore di isteresi.

```
1    if temperatureThresholdEnabled and temperatura > temperatureActiveTh:
        send_alert("Attenzione: temperatura alta")
        temperatureThresholdEnabled = False

    if not temperatureThresholdEnabled and temperatura <= temperatureResetTh:
6        temperatureThresholdEnabled = True

    if pressureThresholdEnabled and pressione > pressureActiveTh:
        send_alert("Attenzione: pressione alta")
        pressureThresholdEnabled = False

11    if not pressureThresholdEnabled and pressione <= pressureResetTh:
        pressureThresholdEnabled = True

    if lightThresholdEnabled and luminosita > lightActiveTh:
16        send_alert("Attenzione: luminosita alta")
        lightThresholdEnabled = False
```

```
if not lightThresholdEnabled and luminosita <= lightResetTh:
    lightThresholdEnabled = True
```

Listing 22: Controllo valori

Per l'invio dell'alert è stata implementata una funzione `send_alert()` a cui viene passato il messaggio da inviare all'utente. Per poter inviare lo stesso messaggio a tutti gli utenti iscritti al bot vengono estratti tutti i chat ID presenti nella tabella del database relativa agli utenti registrati.

Per fare ciò è stato necessario gestire in maniera asincrona l'esecuzione del bot, mediante la libreria **asyncio**.

In questo modo, per ogni chat id selezionato dalla tabella viene generato un loop in cui si richiama la funzione di invio messaggio definita all'interno dello script `telegram_bot`.

```
def send_alert(message):

    query = "SELECT chat_id FROM chatids"
    cursor.execute(query)
    results = cursor.fetchall()
    for result in results:
        print(result)
        asyncio.set_event_loop(asyncio.new_event_loop())
        loop = asyncio.get_event_loop()
        loop.run_until_complete(telegram_bot.invia_messaggio_telegram(
10 message, result, data_config["TELEGRAM_TOKEN"]))
        loop.close()
```

Listing 23: Funzione invio alert

La funzione `invia_messaggio_telegram()` viene implementata all'interno dello script `telegram_bot.py` e riceve come parametri il messaggio che deve essere inviato, il chat id ed il token. Dopo aver associato il token alla variabile `bot`, si invia in modo asincrono il messaggio al chat id passato, tramite la funzione `send_message()`.

```
async def invia_messaggio_telegram(messaggio, ids, token):
    # bot = telegram.Bot(token=data_config["TELEGRAM_TOKEN"])
    bot = telegram.Bot(token=token)

    5 async with bot:
        print("Prova invio messaggio bot.")
        id = str(ids).replace(",", "")
        id_f = str(id).replace("(", "")
        id_s = str(id_f).replace(")", "")
10        await bot.send_message(chat_id=id_s, text=messaggio)
```

Listing 24: Funzione send message bot

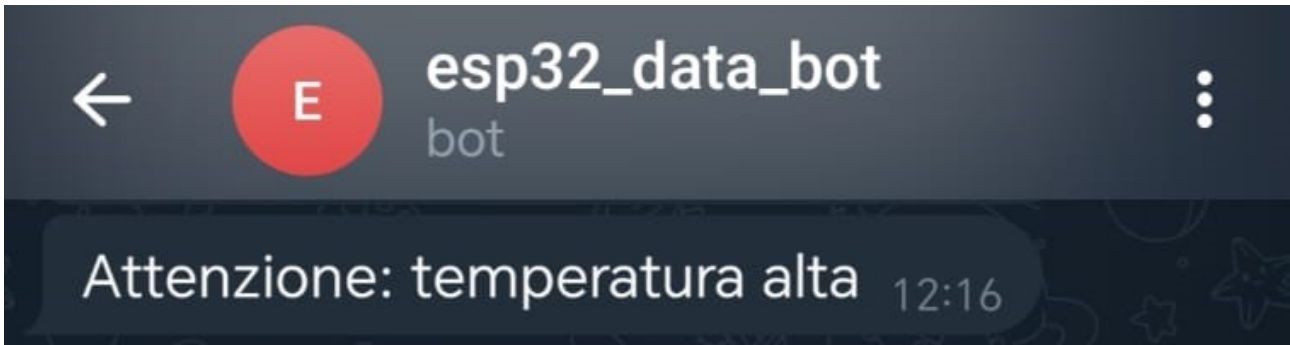


Figura 18: Messaggio di errore

Inviando sulla chat Telegram il comando `/start`, verrà visualizzato il messaggio di benvenuto con la richiesta di inserimento informazioni per ottenere i dati.

Inoltre, non appena l'utente invia il comando, viene preparata una query per cercare all'interno della tabella `chatids` una corrispondenza con il chat id.

Se non esiste nessuna corrispondenza, viene inserito il nome utente ed il relativo chat id.

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    """Send a message when the command /start is issued."""
    user = update.effective_user
    user_id = user.id

    query_chat = "SELECT * FROM chatids WHERE chat_id=%s"
    cursor.execute(query_chat, (user_id,))
    results = cursor.fetchall()

    if results:
        print(f"L'utente {user.full_name} [{user_id}] ha inviato il comando start.")
    else:
        query = "INSERT INTO chatids (Nome, chat_id) VALUES (%s, %s)"
        cursor.execute(query, (user.full_name, user_id))
        conn.commit()
        print("Utente inserito correttamente:")

        await update.message.reply_text(
            f"Ciao {user.full_name}, inserisci /leggi+data+ora+topic, per
            ottenere i dati!",
        )
```

Listing 25: Comando start

Inviando nella chat il comando `/leggi`, seguito da data, ora e topic, l'utente otterrà come risposta l'elenco dei dati memorizzati nel database o nel caso in cui il formato inserito non sia valido, verrà inviato all'utente un messaggio di errore comunicando il formato corretto da utilizzare per la richiesta.



Figura 19: Messaggio di errore

3 Sviluppo Progetto

Per estrarre i dati dal database viene effettuata un'interrogazione di esso mediante una query in cui si vanno a selezionare dalla tabella `sensor_data` i valori relativi alla data e all'ora richiesta dall'utente, attraverso la funzione `cursor.fetchall()`.

Se esiste una corrispondenza dei dati con la data e l'ora richieste, verranno inviati al bot Telegram rispettivamente i valori di temperatura, pressione e luminosità. Altrimenti, verrà generato un messaggio di errore o verrà comunicato che non esiste alcun dato nella data e ora richieste.

```
async def leggi(update, context):
    user = update.effective_user

    if len(context.args) < 2:
        await context.bot.send_message(
            chat_id=user.id,
            text="Devi specificare una data e un'ora nel formato 'YYYY-MM-DD
            HH:MM:SS' e un topic.",
        )
        return

    print("Valore di context.args[0]:", context.args[0])
    print("Valore di context.args[1]:", context.args[1])

    try:
        datetime_str = context.args[0] + " " + context.args[1]
        datetime_filter = datetime.strptime(datetime_str, "%Y-%m-%d %H:%M:%S
        ")

        query = "SELECT * FROM sensor_data WHERE DATE(data) >= %s AND TIME(
        ora) >= %s"
        cursor.execute(query, (datetime_filter.date(), datetime_filter.time
        ()))
        results = cursor.fetchall()

        if results:

            timestamps = [
                datetime.strptime(
                    row[4].strftime("%Y-%m-%d")
                    + " "
                    + (datetime.min + row[5]).strftime("%H:%M:%S"),
                    "%Y-%m-%d %H:%M:%S",
                )
                for row in results
            ]
            pressione = [row[2] for row in results]
            luminosita = [row[3] for row in results]
            temperatura = [row[1] for row in results]

            valori = len(pressione)

            # Calcola media, minimo e massimo per ciascun parametro
            pressione_media = np.mean(pressione)
            pressione_minimo = np.min(pressione)
            pressione_massimo = np.max(pressione)

            luminosita_media = np.mean(luminosita)
            luminosita_minimo = np.min(luminosita)
            luminosita_massimo = np.max(luminosita)
```

```
temperatura_media = np.mean(temperatura)
temperatura_minimo = np.min(temperatura)
50 temperatura_massimo = np.max(temperatura)

message = f"Temperatura media [C]: {temperatura_media},
Pressione media [Pa]: {pressione_media}, Luminosit media [Lumen]: {
luminosita_media}, numero valori: {valori}"
await context.bot.send_message(chat_id=user.id, text=message)
```

Listing 26: Comando leggi con estrazione dati

```
else:
    await context.bot.send_message(
        chat_id=user.id,
        text="Nessun valore trovato per la data e l'ora specificate."
    ),
except ValueError:
    await context.bot.send_message(
        chat_id=user.id,
        text="Formato data e ora non valido. Utilizza 'YYYY-MM-DD HH:MM:
9 SS'.",
    )
```

Listing 27: Comando leggi con estrazione dati

3.9 Grafici

L'utente può richiedere la generazione dei grafici a partire da una determinata data ed ora.

Viene effettuata l'estrazione dei dati dal database per ogni topic, come fatto in precedenza per il comando **leggi** e si calcola per ogni misura il valore massimo, il valore minimo e la media dei valori, tramite le apposite funzioni **max()**, **min()** e **mean()**, del periodo selezionato.

Per ogni topic verrà realizzato un grafico, contenente i valori memorizzati nel database a partire dalla data e ora richiesta dall'utente, mostrando anche le statistiche calcolate.

Una volta generato il grafico, viene salvato in modo temporaneo come immagine, inviato al bot Telegram tramite l'apposita funzione `context.bot.send_photo()` e viene poi eliminato il file.

```
def crea_grafico(
    timestamps, valori, parametro, media, minimo, massimo,
    nome_file
):
    plt.figure(figsize=(10, 6))
    5 plt.plot(timestamps, valori, label=parametro, marker="o")
    plt.axhline(
        y=media, color="blue", linestyle="--", label=f"Media: {
media:.2f}"
    )
    10 plt.axhline(
        y=minimo,
        color="green",
        linestyle="--",
        label=f"Minimo: {minimo:.2f}",
    )
    15 plt.axhline(
```

```

        y=massimo,
        color="red",
        linestyle="--",
        label=f"Massimo: {massimo:.2f}",
20    )

    plt.legend(loc="upper left")
    plt.xlabel("Data e Ora")
    plt.ylabel("Valore")
25    plt.title(f"{parametro} nel Tempo")

    # Aggiungi le statistiche sul lato del grafico
    plt.annotate(
        f"Media: {media:.2f}\nMinimo: {minimo:.2f}\nMassimo: {
massimo:.2f}",
30        xy=(1.05, 0.5),
        xycoords="axes fraction",
        fontsize=12,
        ha="left",
        va="center",
35    )

    # Nascondi l'asse X
    plt.gca().xaxis.set_visible(False)

40    plt.tight_layout()
    plt.savefig(nome_file)
    plt.close()

    # Creare e salvare i grafici
45    grafici = [
        (
            timestamps,
            temperatura,
            "Temperatura [ C ]",
            temperatura_media,
            temperatura_minimo,
            temperatura_massimo,
            "/tmp/grafico_temperatura.png",
50        ),
        (
            timestamps,
            pressione,
            "Pressione [Pa]",
            pressione_media,
            pressione_minimo,
            pressione_massimo,
            "/tmp/grafico_pressione.png",
55        ),
        (
            timestamps,
            luminosita,
            "Luminosit [Lumen]",
            luminosita_media,
            luminosita_minimo,
            luminosita_massimo,
            "/tmp/grafico_luminosita.png",
60        ),
    ],
65    ],
70    ]
```

```

75         for (
            timestamps,
            valori,
            parametro,
            media,
80            minimo,
            massimo,
            nome_file,
        ) in grafici:
            crea_grafico(
15            timestamps, valori, parametro, media, minimo, massimo,
            nome_file
                )

        # Invia i grafici al bot Telegram
        for _, _, _, _, _, _, nome_file in grafici:
90            await context.bot.send_photo(
                chat_id=user.id, photo=open(nome_file, "rb")
            )
            os.remove(nome_file)

```

Listing 28: Calcolo statistiche

Per poter gestire correttamente tutte le funzionalità, all'interno del **main** devono essere richiamate le funzioni per la gestione dei comandi, per l'avvio del bot e per la connessione del database.

Quindi, prima di tutto si crea l'applicazione Telegram tramite la funzione

```
application = Application.builder().token(TOKEN).build()
```

a cui viene passato il token identificativo del bot.

Per consentire la gestione dei comandi `/start` e `/leggi` si inserisce la funzione `add_handler()` che prende come argomenti il comando e la funzione di callback.

```
application.add_handler(CommandHandler("start", start))
application.add_handler(CommandHandler("leggi", leggi))
```

Dopodiché si inizializza e si avvia il bot Telegram tramite la funzione `run_polling()`:

```
application.run_polling(allowed_updates=Update.ALL_TYPES)
```

```
if __name__ == "__main__":

    main_dir = os.path.dirname(__file__)
    config_path = os.path.join(main_dir, "Configurazione.json")

    with open(config_path) as f:
        data_config = json.load(f)

    db_config = {
        "host": data_config["db_config"]["host"], # Indirizzo del server
        "user": data_config["db_config"]["user"], # Nome utente del
        "password": data_config["db_config"]["password"], # Password del
        "database": data_config["db_config"]["database"], # Nome del
    }
```

```
14     }

    topics = ["sensori"] # Un unico topic per i dati strutturati
    # Connessione al database MySQL
    conn = mysql.connector.connect(**db_config)
19    cursor = conn.cursor()
    cursor.execute("SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED")
    print("Connessione al database MySQL avvenuta con successo")

    # Crea un'applicazione Telegram
24    application = Application.builder().token(data_config["TELEGRAM_TOKEN"])
    .build()
    application.add_handler(CommandHandler("start", start))
    application.add_handler(CommandHandler("leggi", leggi))

    application.run_polling(allowed_updates=Update.ALL_TYPES)
```

Listing 29: Definizione istruzioni main

Di seguito si possono vedere degli esempi dei grafici ottenuti:



Figura 20: Richiesta comando leggi

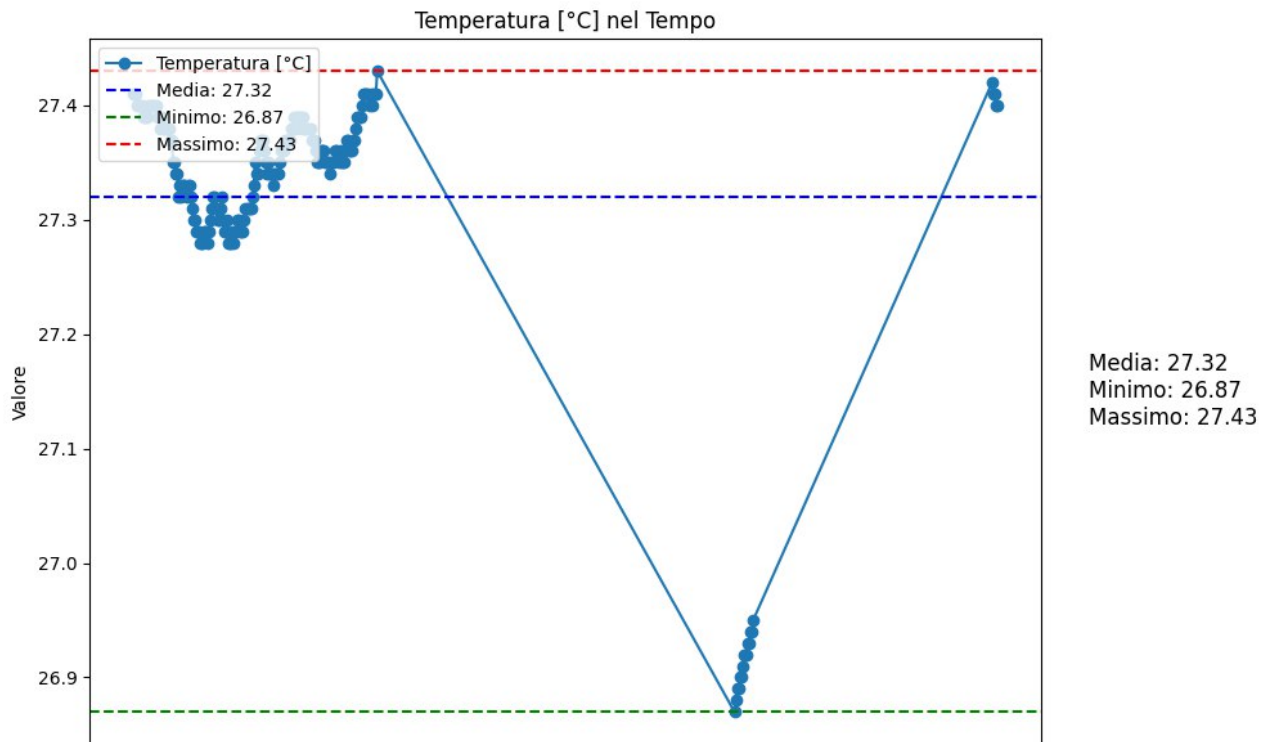


Figura 21: Grafico andamento temperatura

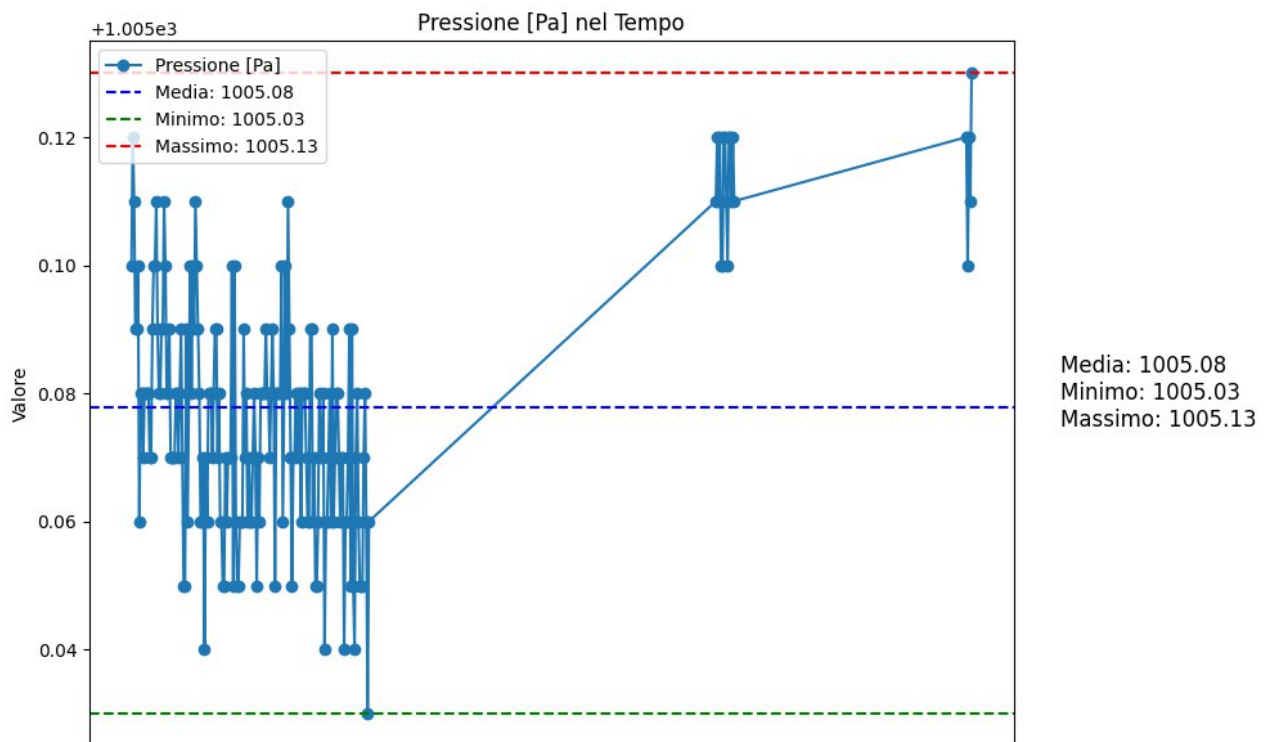


Figura 22: Grafico andamento pressione

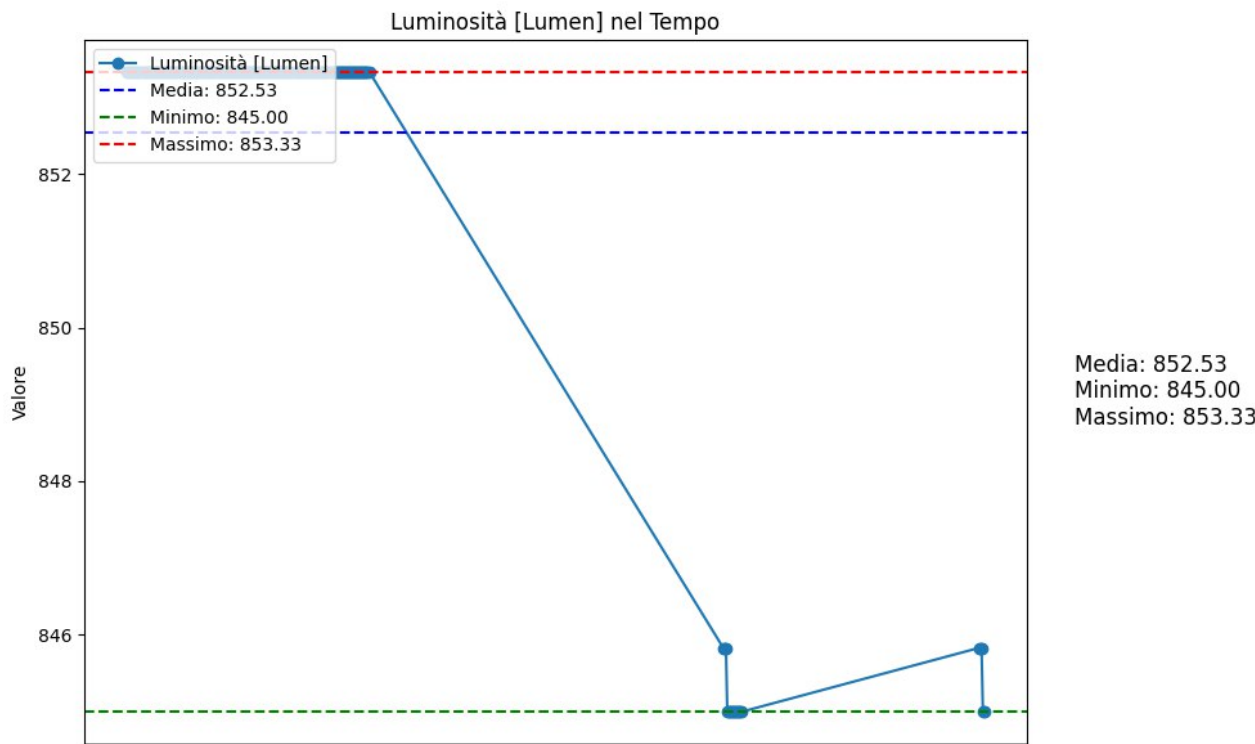


Figura 23: Grafico andamento luminosità

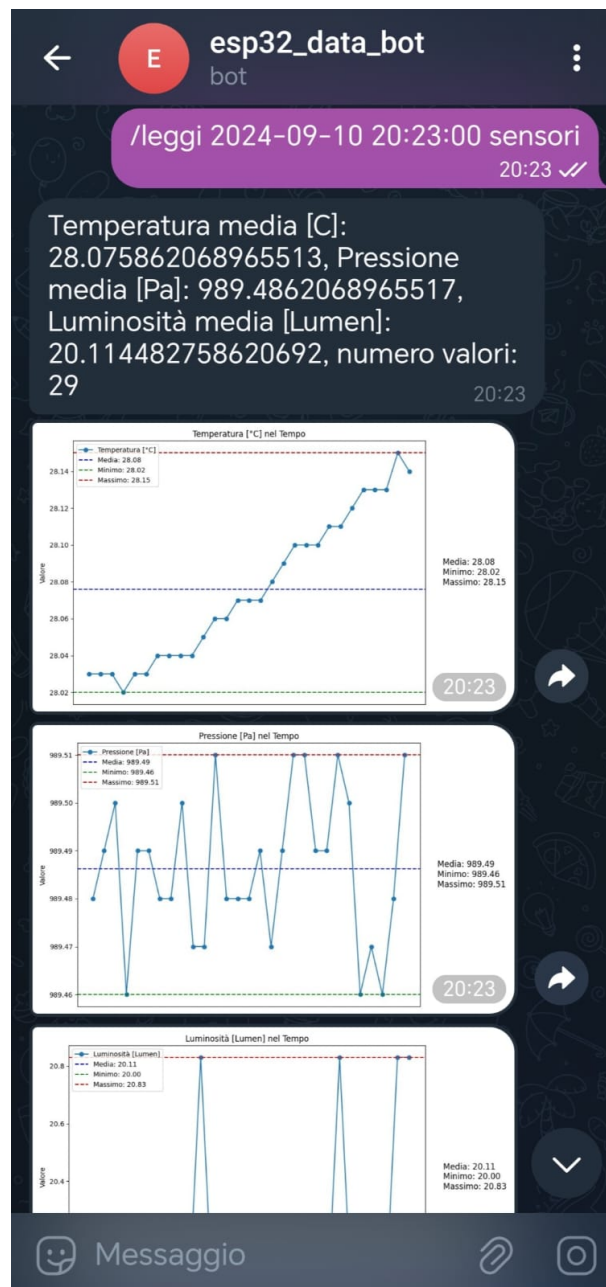


Figura 24: Richiesta comando leggi

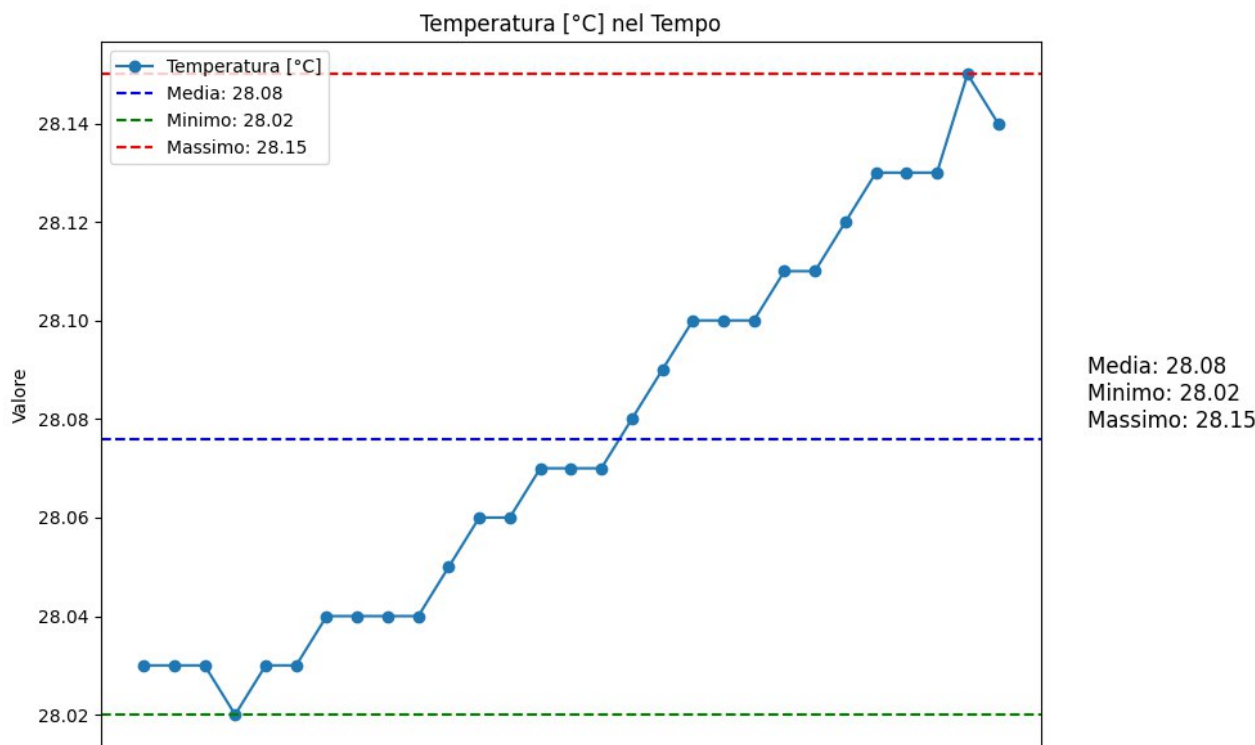


Figura 25: Grafico andamento temperatura

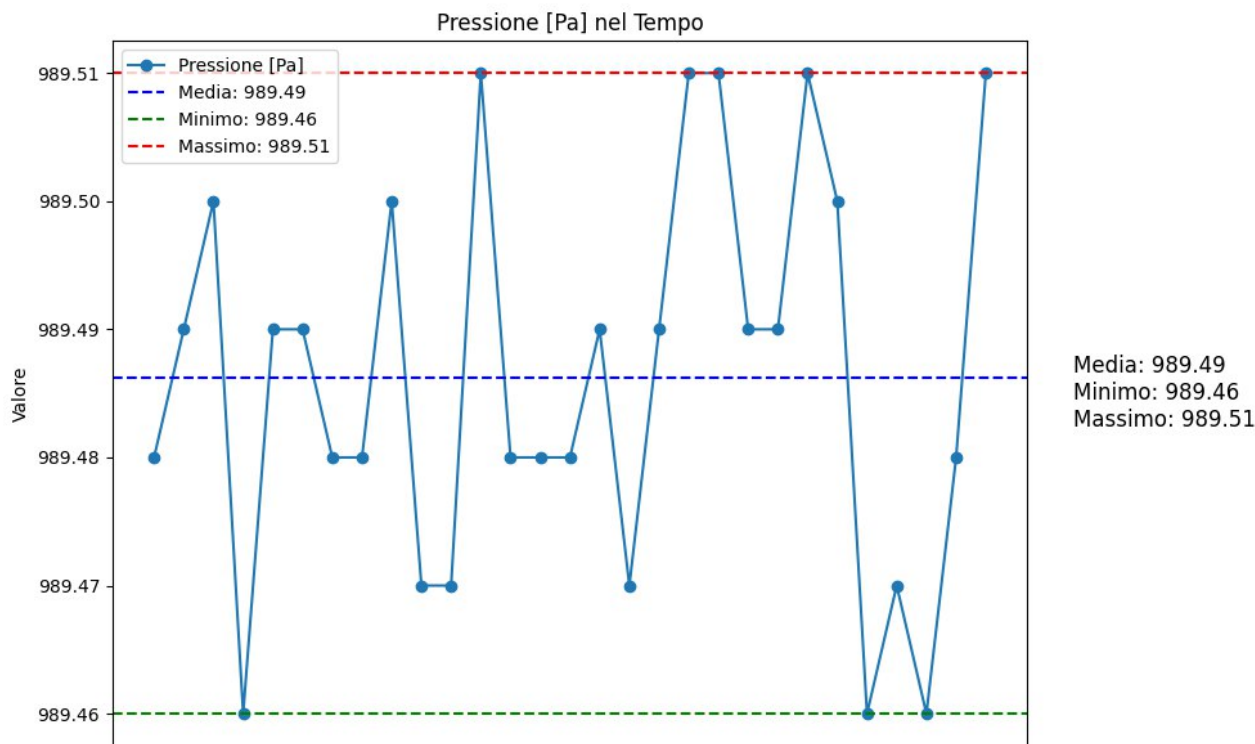


Figura 26: Grafico andamento pressione

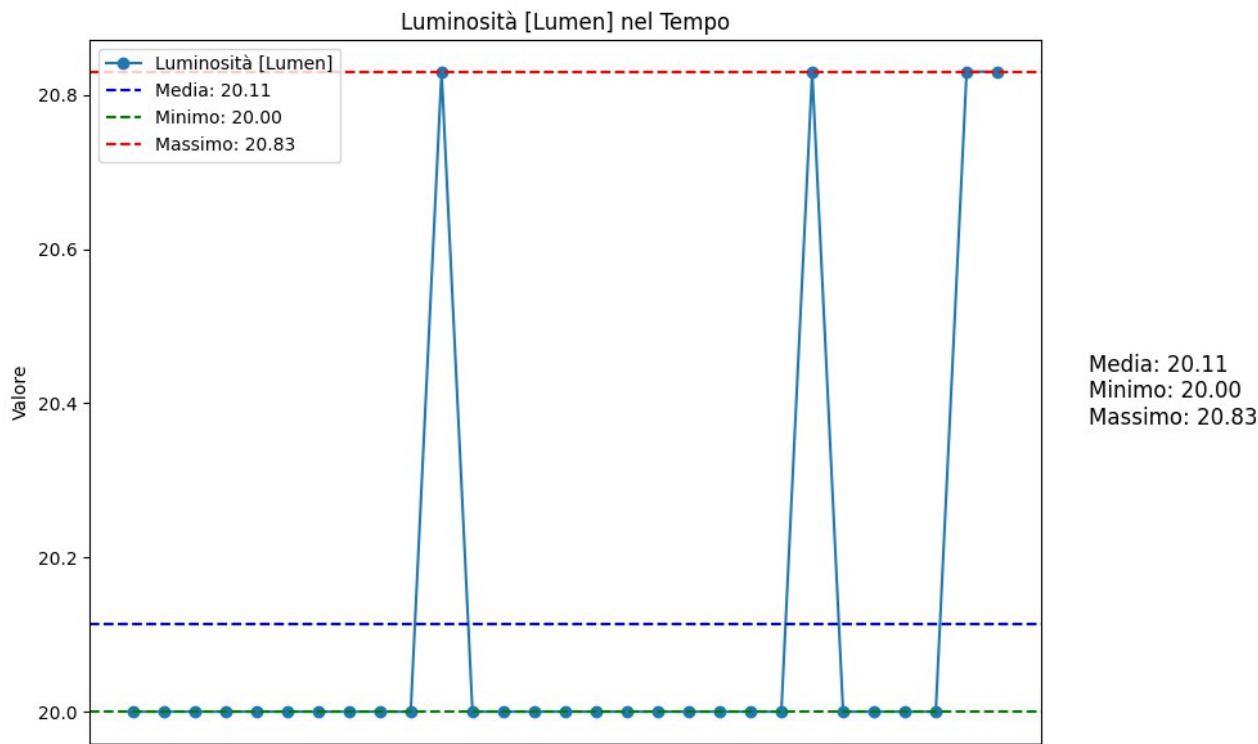


Figura 27: Grafico andamento luminosità

4 Considerazioni finali

Durante lo sviluppo del progetto non abbiamo riscontrato difficoltà rilevanti.

In particolare, studiando i vari sensori, abbiamo notato che il sensore BMP280 non rileva l'umidità, ma solo temperatura e pressione.

Per permettere all'utente di inviare richieste dal bot Telegram, avevamo previsto l'utilizzo della libreria UniversalBotTelegram, ma in fase di test, nonostante la richiesta venisse inviata correttamente alle API di Telegram, non riuscivamo ad ottenere una risposta.

Si è scelto allora di sviluppare il bot lato Python in modo asincrono.

Si è scelto di implementare il database su una macchina Windows, in quanto sul Raspberry abbiamo riscontrato problemi con i permessi Root.

Infine, avendo posizionato i sensori all'interno di una stanza, non siamo riusciti ad ottenere valori che si discostano molto tra loro.

Con questo progetto è stato realizzato un sistema che permette all'utente di monitorare l'ambiente circostante, controllando i dati rilevati dai vari sensori. L'utilità e le funzionalità fornite, integrate con altri dispositivi, possono essere impiegate quotidianamente sia a casa che negli ambienti lavorativi.